Metodologie di Programmazione (M-Z)

II semestre - a.a. 2022 – 2023

Parte 2 – Classi e Oggetti**

a cura di Mattia Samory*



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

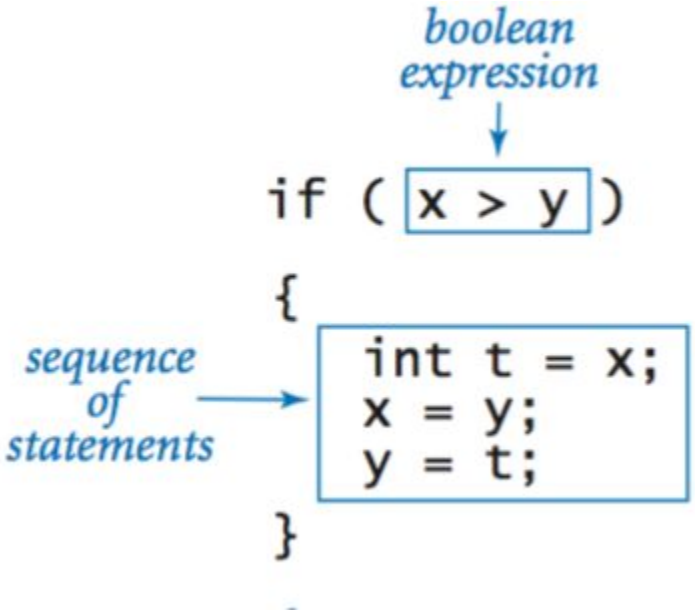
** I crediti sulle slide di questo corso sono riportati nell'ultima slide

📚Letture

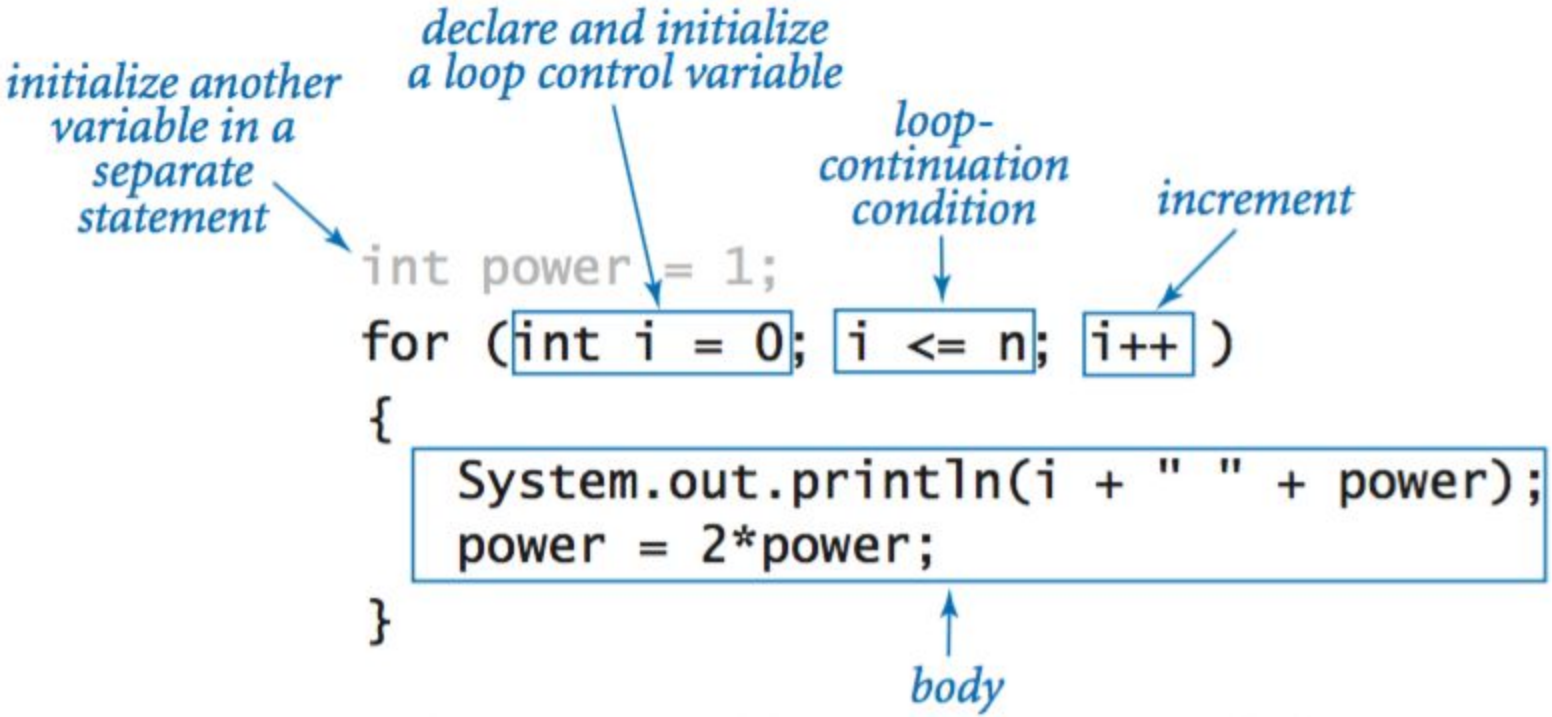
- implementazione di classi (Horstmann capitolo 3)
- documentazione Java (Horstmann capitolo 2.6)
- responsabilità delle classi (Horstmann capitolo 12.1)
- Java cheatsheet https://introcs.cs.princeton.edu/java/11cheatsheet/

🔄 Recap

- if/else
- loop




```
if      (income <      0) rate = 0.00;
else if (income <   8925) rate = 0.10;
else if (income <  36250) rate = 0.15;
else if (income <  87850) rate = 0.23;
else if (income < 183250) rate = 0.28;
else if (income < 398350) rate = 0.33;
else if (income < 400000) rate = 0.35;
else                      rate = 0.396;
```


initialize another variable in a separate statement
declare and initialize a loop control variable
loop-continuation condition
increment
int power = 1;
for (int i = 0; i <= n; i++)
{
System.out.println(i + " " + power);
power = 2*power;
}
body

🎯Obiettivi

- incapsulamento
- uml
- metodi statici vs. d’istanza
- heap, stack, metaspace
- package

Incapsulamento

Incapsulamento

- Perché utilizziamo le parole chiave public e private?
- Per nascondere le informazioni (“information hiding”) all’utente

- Il processo che nasconde i dettagli realizzativi, rendendo pubblica un’interfaccia, prende il nome di incapsulamento
  - Dettagli realizzativi: campi e implementazione
  - Interfaccia pubblica: metodi pubblici

Perché incapsulare?

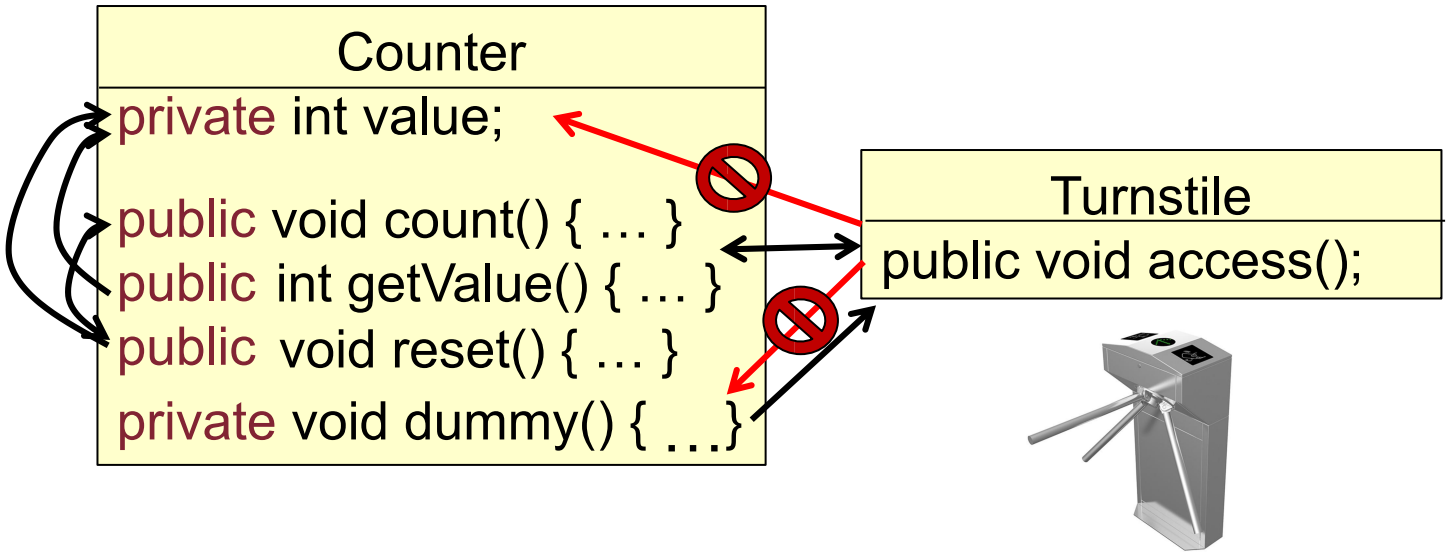
- Si semplifica e modularizza il lavoro di sviluppo assumendo un certo funzionamento a “scatola nera”
- Non è necessario sapere tutto, soprattutto molti inutili dettagli
- L’incapsulamento facilita il lavoro di gruppo e l’aggiornamento del codice (maintenance)
- Aiuta a rilevare errori: in presenza di moltissime classi, un certo errore si verifica solo in una determinata classe per cui ci si può concentrare su di essa

Come interagiscono le classi tra loro?

- Una classe interagisce con le altre **principalmente (=quasi solo)** attraverso i costruttori e i metodi pubblici
- Le altre classi non devono conoscere i dettagli implementativi di una classe per usarla in modo efficace

Accesso a campi e metodi (inclusi i costruttori)

- I campi e i metodi possono essere pubblici, privati (o protetti, come vedremo più in là)
- I metodi di una classe possono chiamare i metodi pubblici e privati della stessa classe
- I metodi di una classe possono chiamare i metodi pubblici (ma non quelli privati) di altre classi



🎯Obiettivi

- ✅ incapsulamento
- ☐ uml
- ☐ metodi statici vs. d’istanza
- ☐ heap, stack, metaspace
- ☐ package

UML

UML parte 1: i diagrammi delle classi



Accesso non effettuato discussioni contributi registrati entra

Voce | Discussione | Leggi | Modifica | Modifica wikitesto | Cronologia | Cerca in Wikipedia

Class diagram

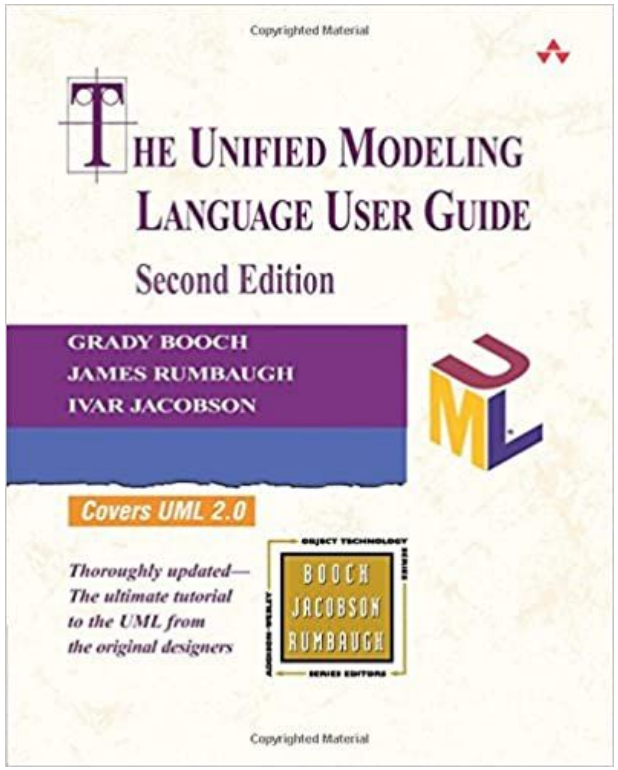
Da Wikipedia, l'enciclopedia libera.

I *diagrammi delle classi* (**class diagram**) sono uno dei tipi di diagrammi che possono comparire in un modello UML. In termini generali, consentono di descrivere *tipi di entità*, con le loro caratteristiche e le eventuali relazioni fra questi tipi. Gli strumenti concettuali utilizzati sono il concetto di classe del paradigma object-oriented e altri correlati (per esempio la generalizzazione, che è una relazione concettuale assimilabile al meccanismo *object-oriented* dell'ereditarietà).

In ingegneria del software, **UML** (*Unified Modeling Language*, "linguaggio di modellizzazione unificato") è un linguaggio di modellazione e di specifica basato sul paradigma orientato agli oggetti. Il nucleo del linguaggio fu definito nel 1996 da Grady Booch, Jim Rumbaugh e Ivar Jacobson (detti "i tre amigos") sotto l'egida dell'Object Management Group (OMG), consorzio che tuttora gestisce lo standard UML.

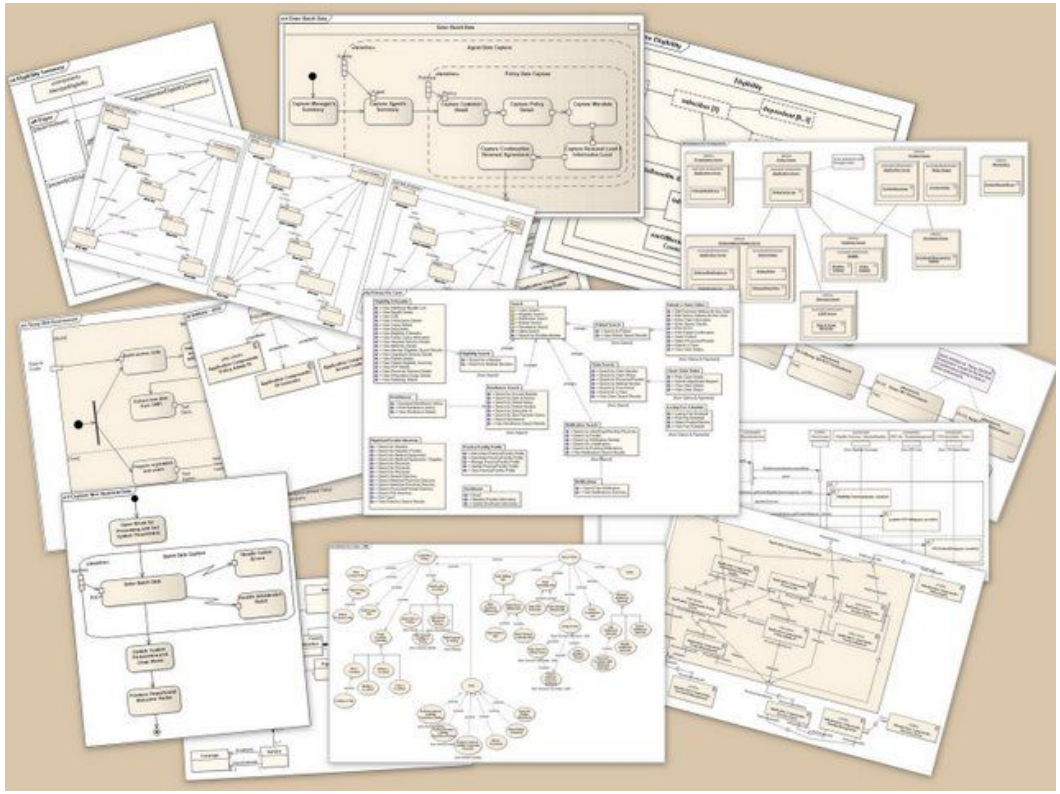


UML parte 1: i diagrammi delle classi



https://personal.utdallas.edu/~chung/Fujitsu/UML_2.0/Rumbaugh--UML_2.0_Reference_CD.pdf

UML parte 1: i diagrammi delle classi



UML parte 1: i diagrammi delle classi

Object Oriented Analysis

Reverse Engineering

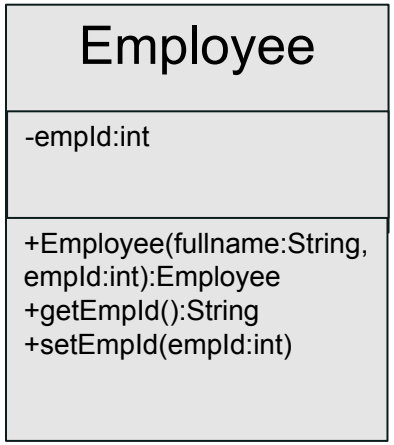
Entità (**Oggetti** nel mondo reale)

Classe in UML **modello**

Classe in Java

Oggetti in Java

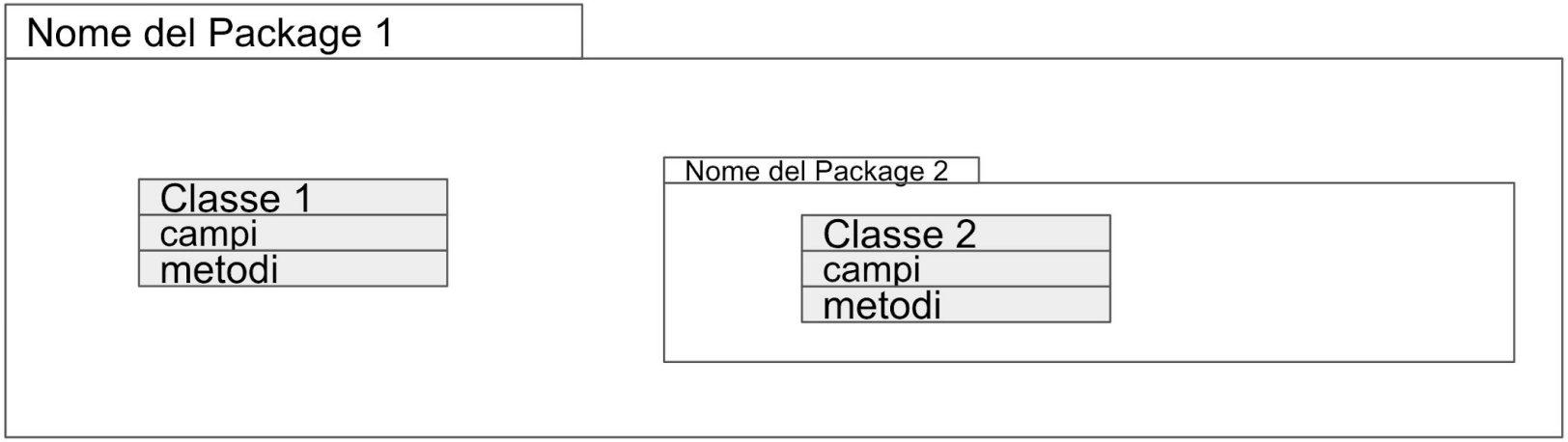




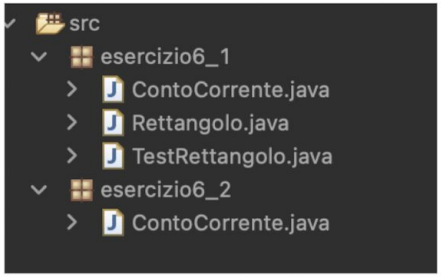

```java
// Java bean for Employee
public class Employee extends Person {
    // Private variable
    private int empId;
    // Constructor
    public Employee(String fullName, int empId) {
        super(fullName);
        setEmpId(empId);
    }
    // Getter and setter for variable
    public int getEmpId() {
        return empId;
    }
    public void setEmpId (int empId) {
        this.empId=empId;
    }
}
```

```
new Employee(“John”,...);
new Employee(“Dessy”,...);
```

UML parte 1: i diagrammi delle classi



In questo Diagramma abbiamo un package che include una classe e un sotto package





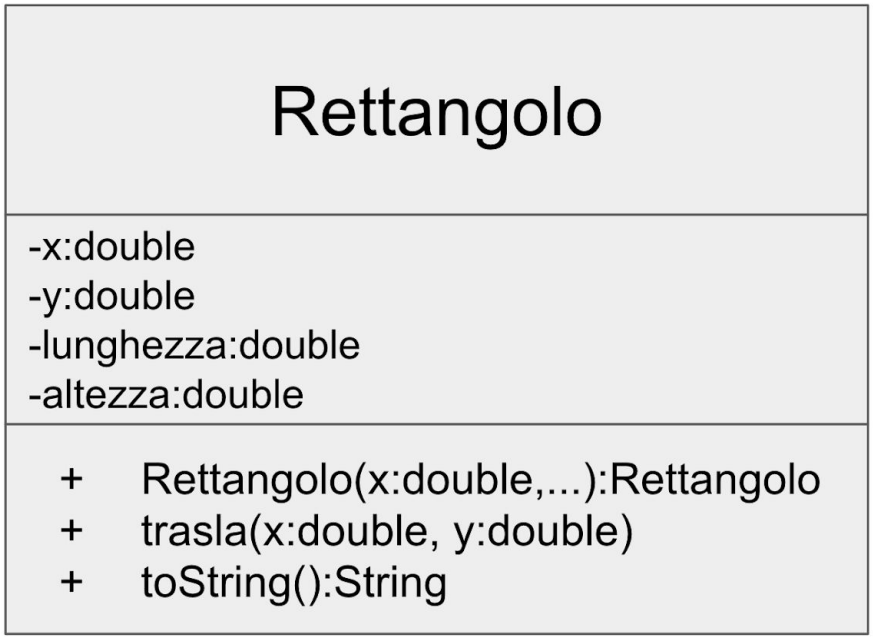
UML parte 1: i diagrammi delle classi

```java
public class Rettangolo {
    private double x, y, lunghezza;
    private double altezza;

    public Rettangolo(double x, double y,double lunghezza, double altezza)
    {
        this.x=x;
        this.y=y;
        this.lunghezza=lunghezza;
        this.altezza=altezza;
    }

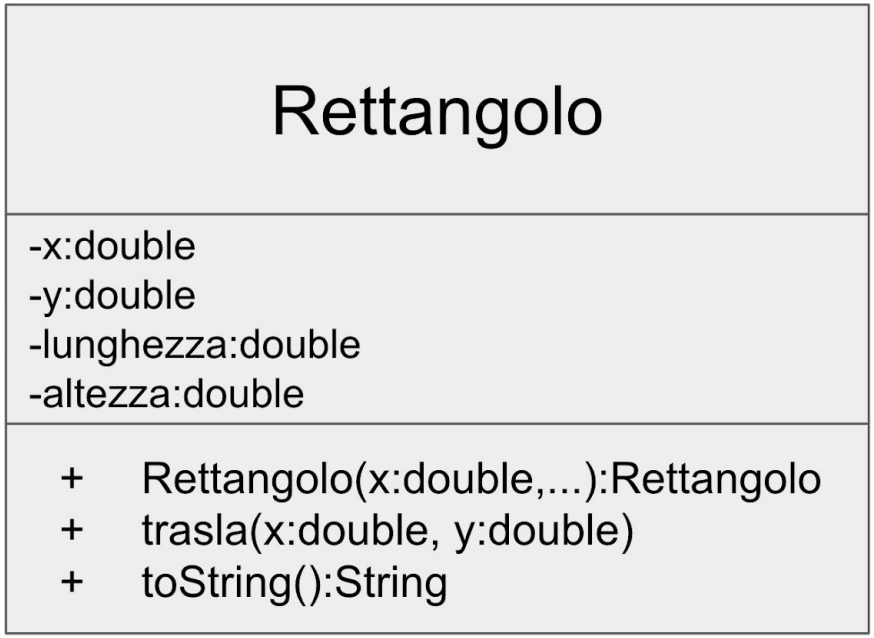
    public void trasla(double x, double y)
    {
        this.x+=x;
        this.y+=y;
    }

    public String toString()
    {
         double x2=x+lunghezza;
         double y2=y+altezza;
         return "("+x+", "+y+")-> ("+x2+", "+y2+")" ;
    }
}
```



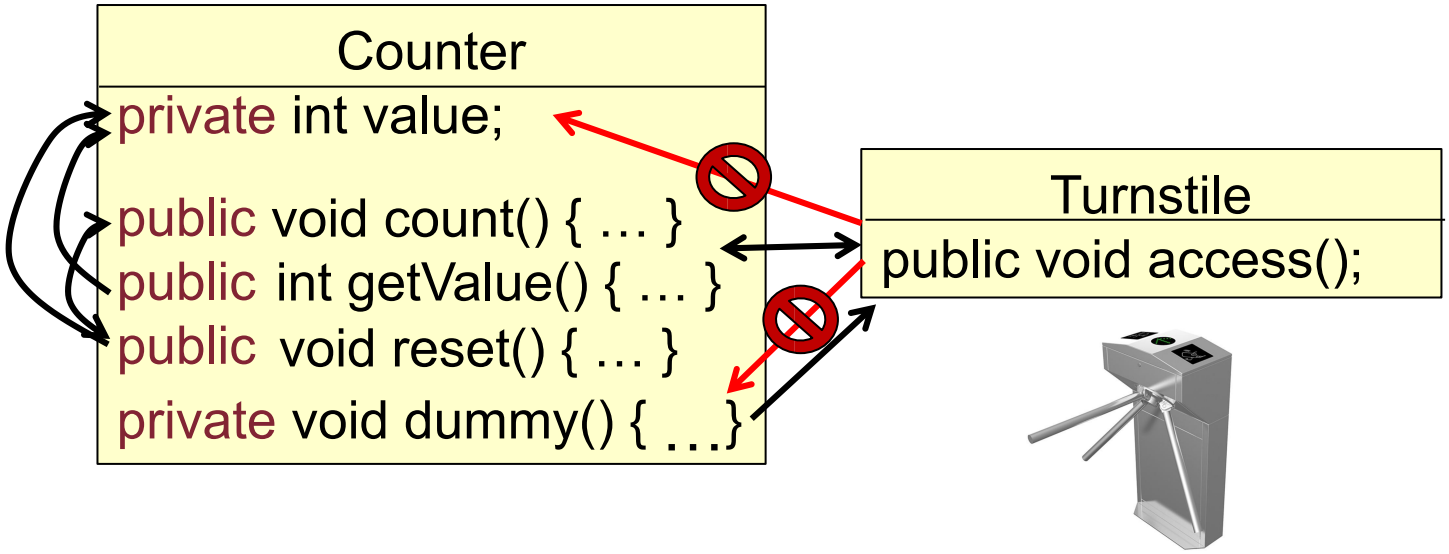
UML parte 1: i diagrammi delle classi

- \+ (visibilità pubblica): ogni elemento che può accedere alla classe può anche accedere a ogni suo membro con visibilità pubblica
- \- (visibilità privata): solo le operazioni della classe possono accedere ai membri con visibilità privata
- # (visibilità protetta): solo le operazioni appartenenti alla classe o ai suoi discendenti possono accedere ai membri con visibilità protetta
- ~ (visibilità package): ogni elemento nello stesso package della classe (o suo sottopackage annidato) può accedere ai membri della classe con visibilità package



Accesso a campi e metodi (inclusi i costruttori)

- I campi e i metodi possono essere pubblici, privati (o protetti, come vedremo più in là)
- I metodi di una classe possono chiamare i metodi pubblici e privati della stessa classe
- I metodi di una classe possono chiamare i metodi pubblici (ma non quelli privati) di altre classi



🎯Obiettivi

- ✅ incapsulamento
- ✅ uml
- ☐ metodi statici vs. d’istanza
- ☐ heap, stack, metaspace
- ☐ package

Design orientato agli oggetti

Un esempio: realizzare un semplice menù

- Dovete realizzare la classe Menu, in grado di visualizzare un menù come questo:

1) Inizia il gioco
2) Carica gioco
3) Aiuto
4) Esci

Fase 1: identifica i metodi richiesti

- Aggiungere una nuova opzione
- Visualizzare il menù

Fase 2: specifica l'interfaccia pubblica

- Aggiungere una nuova opzione:

```
public void addOption(String option) { }
```

- Visualizzare il menù:

```
public void display() { }
```

- Costruire l'oggetto:
  - Costruttore con una prima opzione in input?
  - Costruttore vuoto?
- Meglio evitare casi speciali:

```
public Menu() { }
```

Fase 3: scrivere la documentazione per l'interfaccia pubblica

```java
/**
 * Un menu' che viene visualizzato in una finestra di console
 * @author navigli
 *
 */
public class Menu
{
	/**
	 * Costruisce un menu' vuoto (senza opzioni)
	 */
	public Menu()
	{
	}

	/**
	 * Aggiunge un'opzione alla fine del menu'
	 * @param option l'opzione da aggiungere
	 */
	public void addOption(String option)
	{
	}

	/**
	 * Visualizza il menu' su console
	 */
	public void display()
	{
	}
}
```

Fase 4:Identifica i campi

- In ogni momento, qual è lo stato di un oggetto di tipo Menu?

```
public class Menu
{
        private String menuText;
        private int optionCount;

        ...
}
```

Fase 5:Implementa i metodi

```
public class Menu
{
	...

	/**
	 * Costruisce un menu' vuoto (senza opzioni)
	 */
	public Menu()
	{
		menuText = "";
		optionCount = 0;
	}

	/**
	 * Visualizza il menu' su console
	 */
	public void display()
	{
		System.out.println(menuText);
	}

	/**
	 * Aggiunge un'opzione alla fine del menu'
	 * @param option l'opzione da aggiungere
	 */
	public void addOption(String option)
	{
		optionCount++;
		menuText += optionCount + ") " + option + "\n";
	}
}
```

Fase 6: Collauda la classe

- Scriviamo un'altra classe per "testare" (collaudare) la nostra:

```
public class MenuTest
{
	static public void main(String[] args)
	{
		Menu menu = new Menu();

		menu.addOption("Open new account");
		menu.addOption("Log into existing account");
		menu.addOption("Help");
		menu.addOption("Quit");

		menu.display();
	}
}
```

Esercizio 1/2

- Progettare una classe Programmatore i cui oggetti rappresentano persone che svolgono il lavoro di sviluppo presso un'azienda
- La classe implementa i seguenti metodi:
  - un costruttore con il nome e cognome della persona
  - un metodo setAzienda che imposta il nome dell'azienda per cui la persona lavora
  - un metodo addLinguaggio che aggiunge un linguaggio di programmazione a quelli inizialmente usati dal programmatore
  - i metodi getNome, getCognome e getAzienda che restituiscono i valori corrispondenti
  - un metodo getLinguaggi che restituisce la stringa dei linguaggi (separati da spazio) noti al programmatore

Esercizio 2/2

- Implementare un main all'interno della stessa classe in modo che il seguente codice funzioni correttamente:

```
Programmatore p1 = new Programmatore("Bjarne", "Stroustrup");
Programmatore p2 = new Programmatore("Brian", "Kernighan");
Programmatore p3 = new Programmatore("James", "Gosling");

p1.addLinguaggio("C");
p1.addLinguaggio("C++");
p1.setAzienda("Morgan Stanley");

p2.addLinguaggio("C");
p2.addLinguaggio("AWK");

p3.addLinguaggio("Java");
p3.setAzienda("Oracle");

// stampa: Morgan Stanley
System.out.println(p1.getAzienda());
// stampa: C AWK
System.out.println(p2.getLinguaggi());
```

Esercizio: registratore di cassa

- Progettare una classe che costituisca un modello di registratore di cassa
- La classe deve consentire a un cassiere di inserire i prezzi degli articoli e, data la quantità di denaro pagata dal cliente, calcolare il resto dovuto
- Quale stato (campi) dovete prevedere?
- Quali metodi vi servono?
  - Registra il prezzo di vendita per un articolo
  - Conclude la transazione, calcola il resto dovuto al cliente nel momento in cui effettua il pagamento e restituendolo in uscita

Esercizio: punti e segmenti

- Progettare una classe Punto per la rappresentazione di un punto nello spazio tridimensionale
- E una classe Segmento per rappresentare un segmento nello spazio tridimensionale
- Scrivere una classe di test che crei:
  - due oggetti della classe Punto con coordinate (1, 3, 8) e (4, 4, 7)
  - un oggetto della classe Segmento che rappresenti il segmento che unisce i due punti di cui sopra

Uso della memoria in Java

Riferimenti e oggetti

- Per chi sa cosa siano i puntatori, i riferimenti sono ciò che ne è rimasto in Java
- Un riferimento è un indirizzo di memoria
  - Tuttavia non conosciamo il valore numerico dell'indirizzo
- Quindi gli oggetti non sono mai memorizzati direttamente nelle variabili, ma solo mediante il loro riferimento

Variabili di tipi primitivi e variabili “riferimento”

- Le variabili contengono:
  - valori di tipi di dati primitivi
  - oppure riferimenti a oggetti
- Non esistono variabili che contengono oggetti
  - Analogamente ai puntatori in C

null

- null fa riferimento a nessun oggetto
- Si può assegnare a variabili di tipo oggetto
  - worldGreeter = null;
- Si può testare se un riferimento è null
  - if (worldGreeter == null) . . .
- Dereferenziare (accedere a) null causa NullPointerException

this

riferimento a parametro implicito della chiamata a metodo

Esempio: uguaglianza

```
public boolean equals(Greeter other)
{
   if (this == other) return true;
   return name.equals(other.name);
}
```

Esempio: costruttore

```
public Greeter(String name)
{
   this.name = name;
}
```

Passaggio di parametri

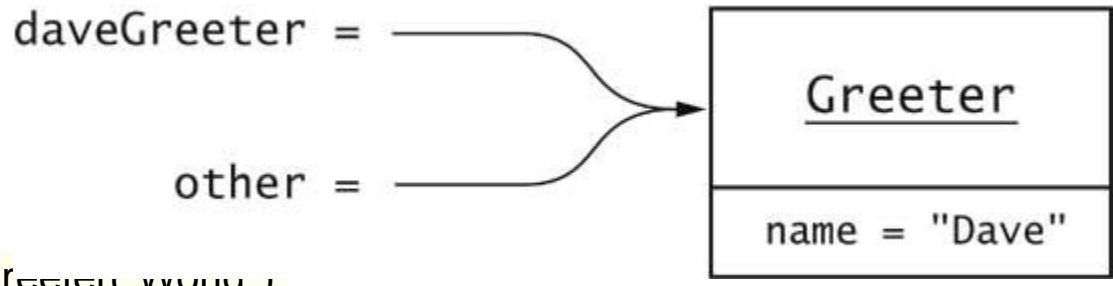
- "chiamata per valore": il metodo riceve una copia del valore
- copiare un riferimento ad oggetto permette di modificarlo

```
public void copyNameTo(Greeter other)

{
   other.name = this.name;
}

...

Greeter worldGreeter = new Greeter("World");
Greeter daveGreeter = new Greeter("Dave");
worldGreeter.copyNameTo(daveGreeter);
```



nessuna chiamata per riferimento

```
public void copyLengthTo(int n)
{
   n = name.length();
}
public void copyGreeterTo(Greeter other)
{
   other = new Greeter(name);
}
```

Nessun effetto per le seguenti chiamate:

```
int length = 0;
worldGreeter.copyLengthTo(length); // length still 0
worldGreeter.copyGreeterTo(daveGreeter) // daveGreeter unchanged
```

Oggetti: i tre passi della dichiarazione, creazione e assegnazione

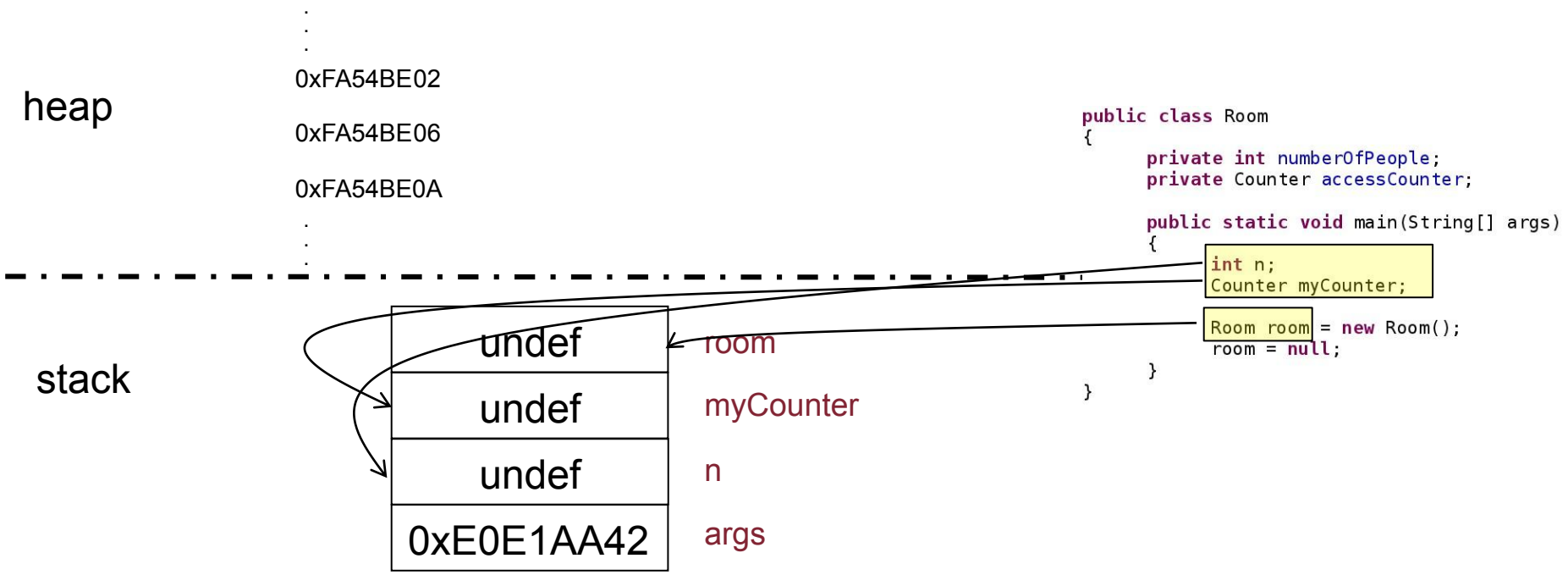
- **Dichiarazione**:
  - **Menu menu** = new Menu();
- **Creazione**:
  - Menu menu = **new Menu();**
- **Assegnazione**:
  - Menu menu **=** new Menu();

Anatomia della memoria

Esistono due tipi di memoria: lo **heap** e lo **stack**

Sullo stack vanno le variabili locali

Sullo heap vanno le aree di memoria allocate per la creazione dinamica

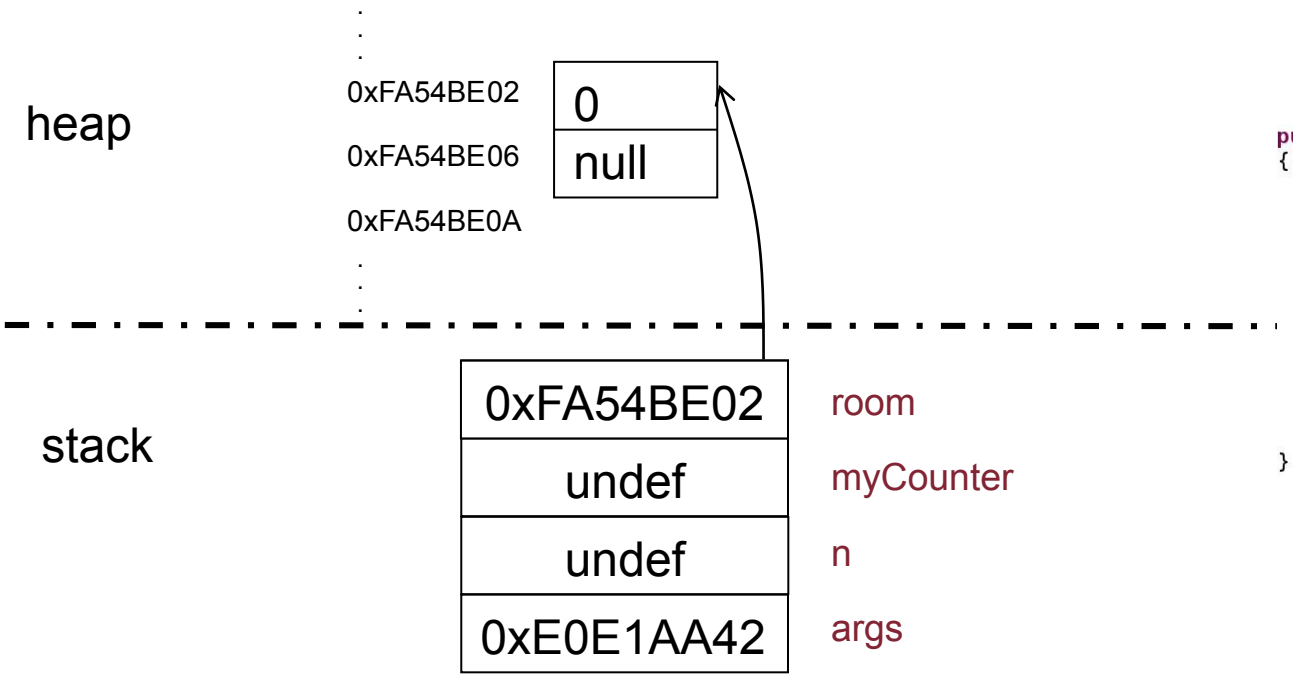


Anatomia della memoria

Esistono due tipi di memoria: lo **heap** e lo **stack**

Sullo stack vanno le variabili locali

Sullo heap vanno le aree di memoria allocate per la creazione dinamica




```
public class Room
{
      private int numberOfPeople;
      private Counter accessCounter;

      public static void main(String[] args)
      {
            int n;
            Counter myCounter;

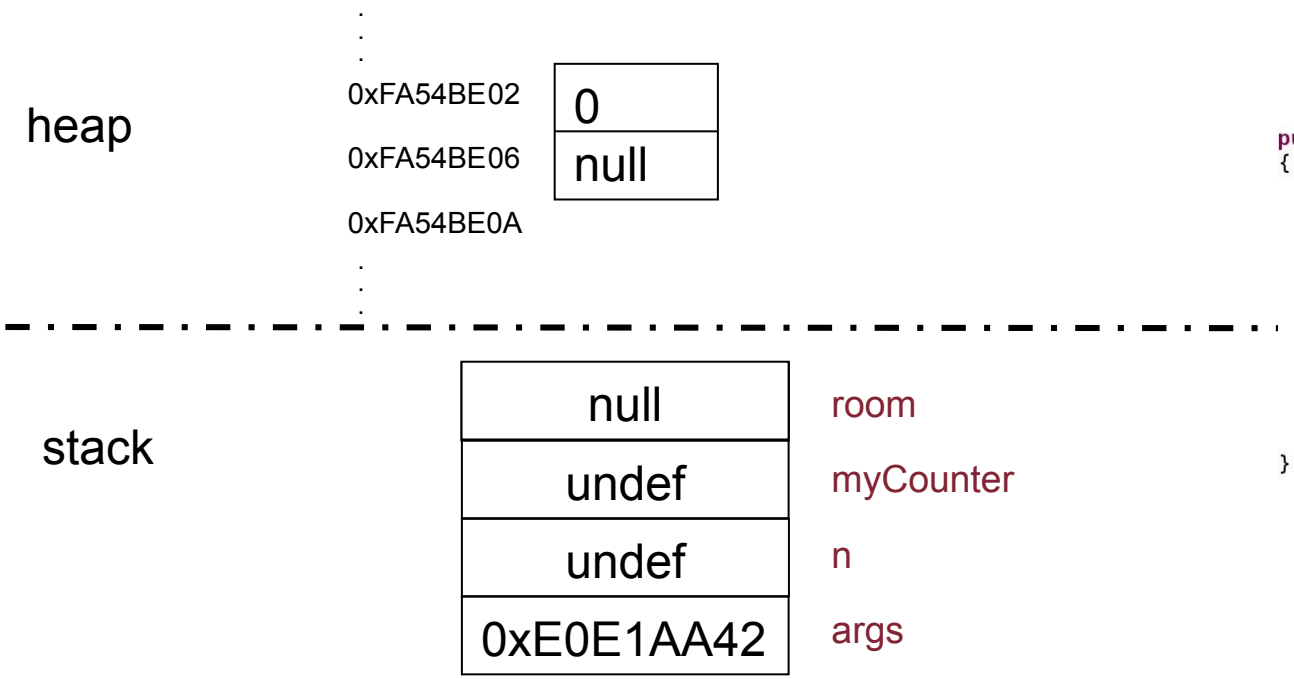
            Room room = new Room();
            room = null;
      }
}
```

Anatomia della memoria

Esistono due tipi di memoria: lo **heap** e lo **stack**

Sullo stack vanno le variabili locali

Sullo heap vanno le aree di memoria allocate per la creazione dinamica




```
public class Room
{
      private int numberOfPeople;
      private Counter accessCounter;

      public static void main(String[] args)
      {
            int n;
            Counter myCounter;

            Room room = new Room();
            room = null;
      }
}
```

Campi di classe: la parola chiave static

- I campi di una classe possono essere dichiarati static
- Un campo static è relativo all'intera classe, **NON** al singolo oggetto istanziato
- Un campo static esiste in una sola locazione di memoria, allocata prima di qualsiasi oggetto della classe in una zona speciale di memoria nativa chiamata MetaSpace
- Viceversa, per ogni campo non static esiste una locazione di memoria **per ogni oggetto**, allocata a seguito dell'istruzione new

Esempio

```
public class Room
{
	static private int totalNumberOfPeople;

	private int numberOfPeople;
	private Counter accessCounter;

	public static void main(String[] args)
	{
		int n;
		Counter myCounter;

		Room room1 = new Room();
		Room room2 = new Room();
	}
}
```

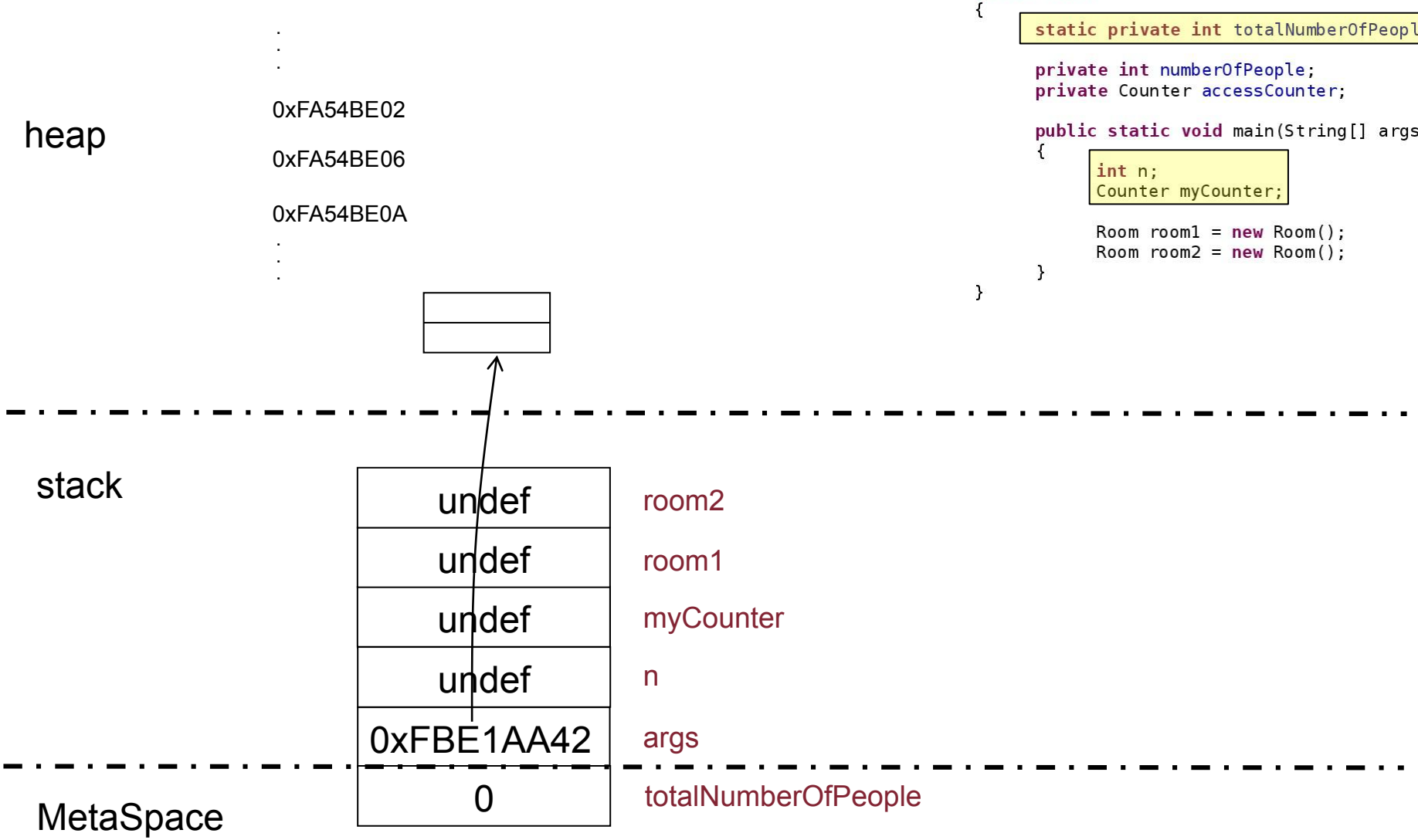
Esempio

```
public class Room
{
      static private int totalNumberOfPeople;

      private int numberOfPeople;
      private Counter accessCounter;

      public static void main(String[] args)
      {
            int n;
            Counter myCounter;

            Room room1 = new Room();
            Room room2 = new Room();
      }
}
```



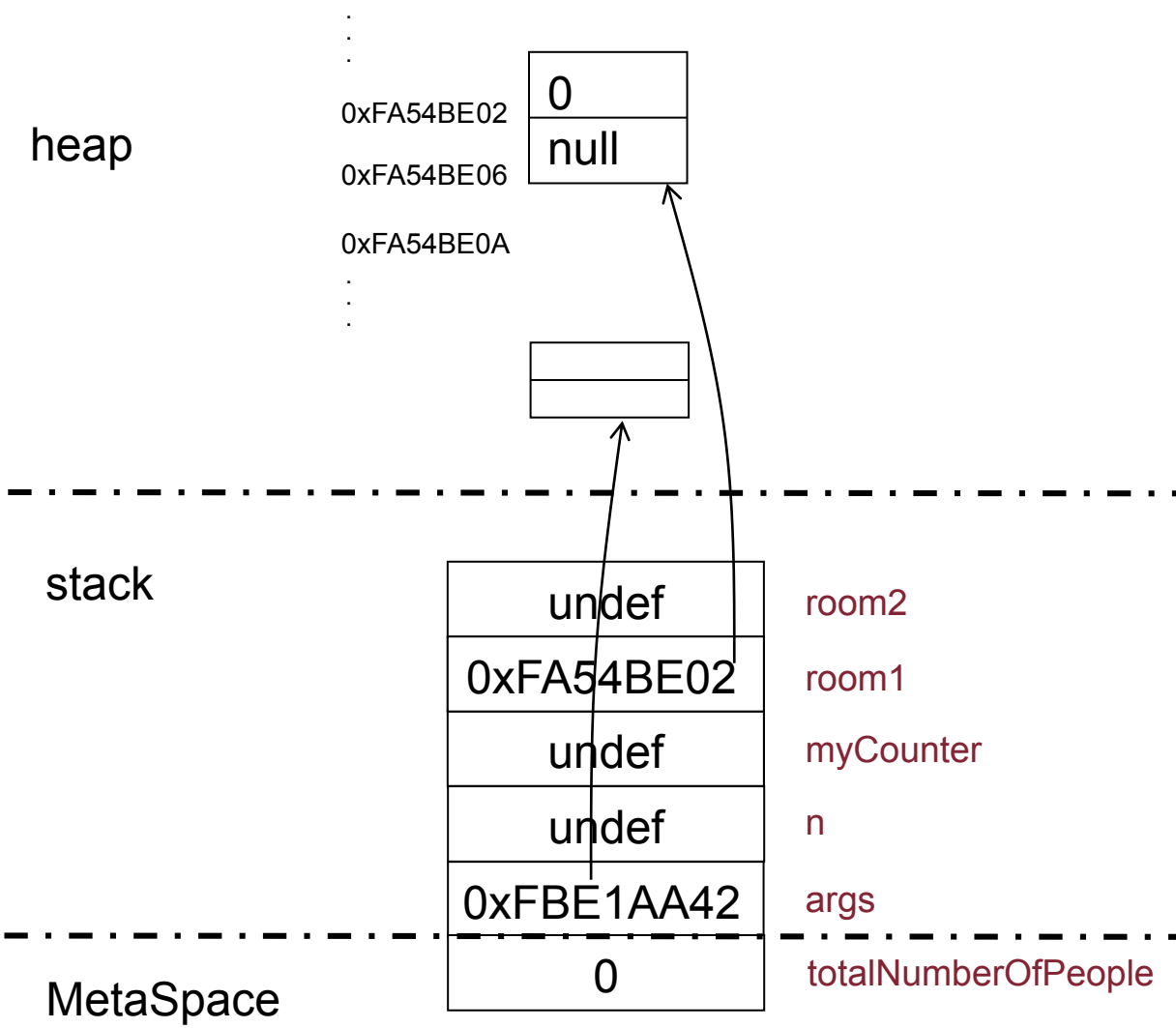
Esempio

```
public class Room
{
     static private int totalNumberOfPeople;

     private int numberOfPeople;
     private Counter accessCounter;

     public static void main(String[] args)
     {
          int n;
          Counter myCounter;

          Room room1 = new Room();
          Room room2 = new Room();
     }
}
```



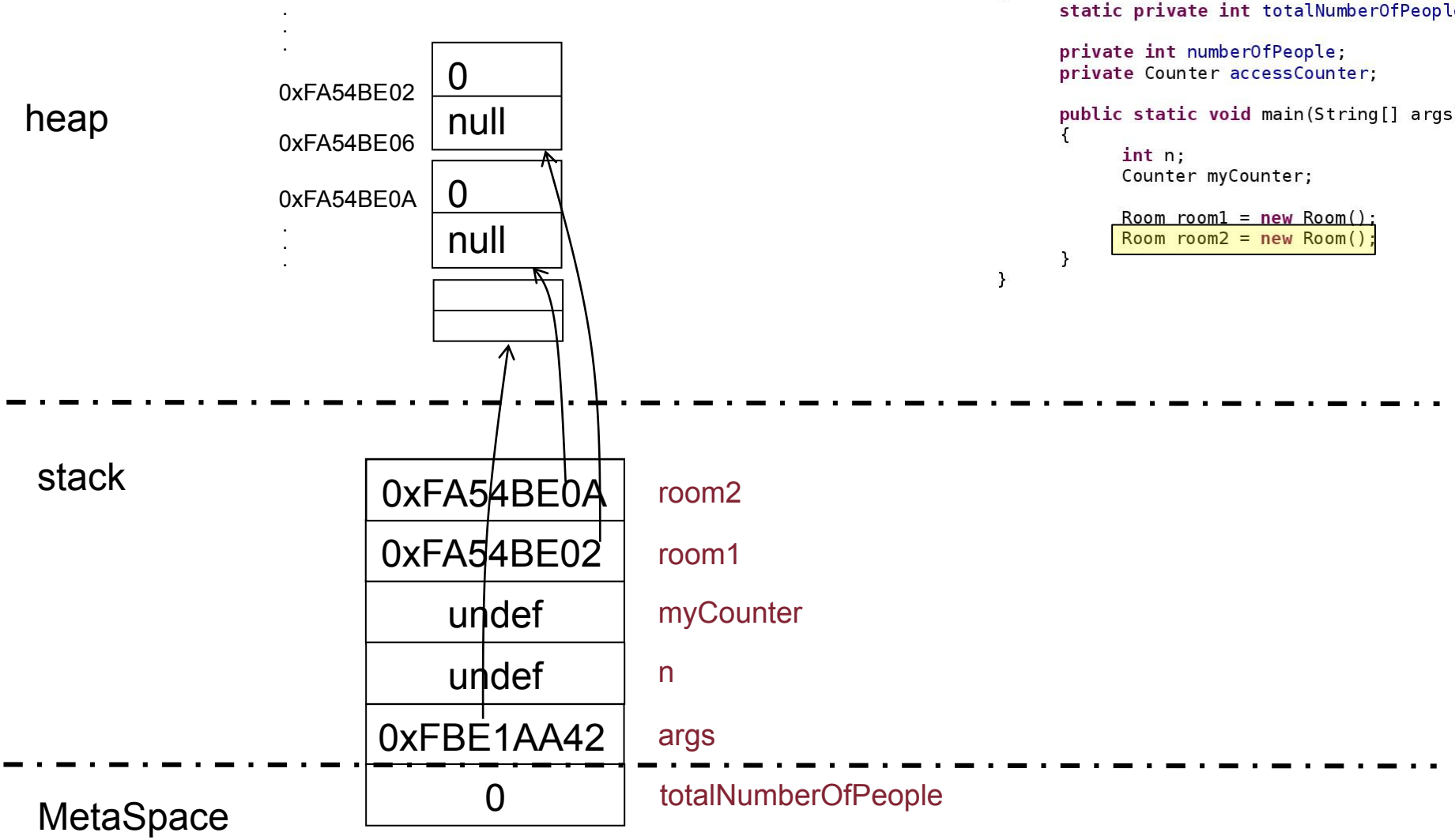
Esempio

```
public class Room
{
      static private int totalNumberOfPeople;

      private int numberOfPeople;
      private Counter accessCounter;

      public static void main(String[] args)
      {
            int n;
            Counter myCounter;

            Room room1 = new Room();
            Room room2 = new Room();
      }
}
```



Esercizio 1.1.2. Disegnare lo stato della memoria (heap e stack) appena prima del termine dell'esecuzione del metodo `main` della classe seguente:

```
public class Tornello
{
  static private int passaggi;

  public void passa() { passaggi++; }
  public static int getPassaggi() { return passaggi; }

  public static void main(String[] args)
  {
    Tornello t1 = new Tornello();
    t1.passa();
    Tornello t2 = new Tornello();
    for (int k = 0; k < 10; k++) t2.passa();
    int g;
    String s = null;
    // fotografa qui lo stato della memoria
  }
}
```


stack
heap
Si rappresentano i
frame di attivazione
(chiamate ai metodi) e
le variabili locali
Si rappresenta
l'allocazione degli
oggetti
metaspace
Si rappresenta
l'allocazione dei campi
statici

Tornello

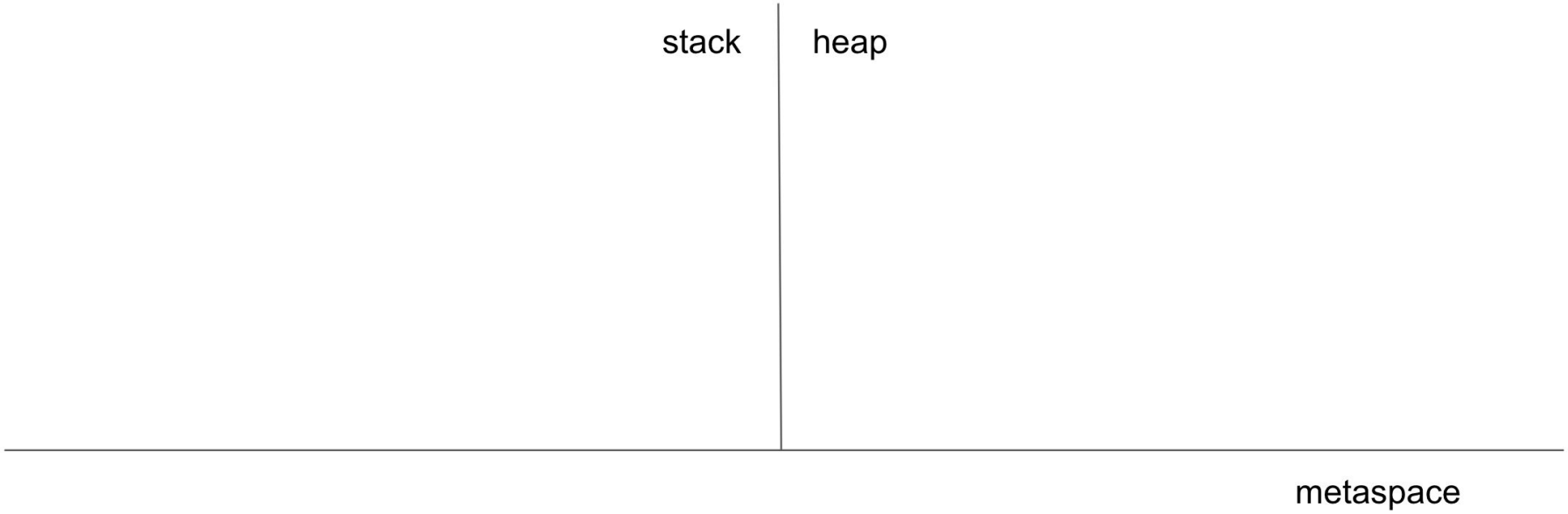


```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```

La prima cosa in assoluto è analizzare i campi statici



Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```

stack | heap

metaspace

passaggi=0

Tornello

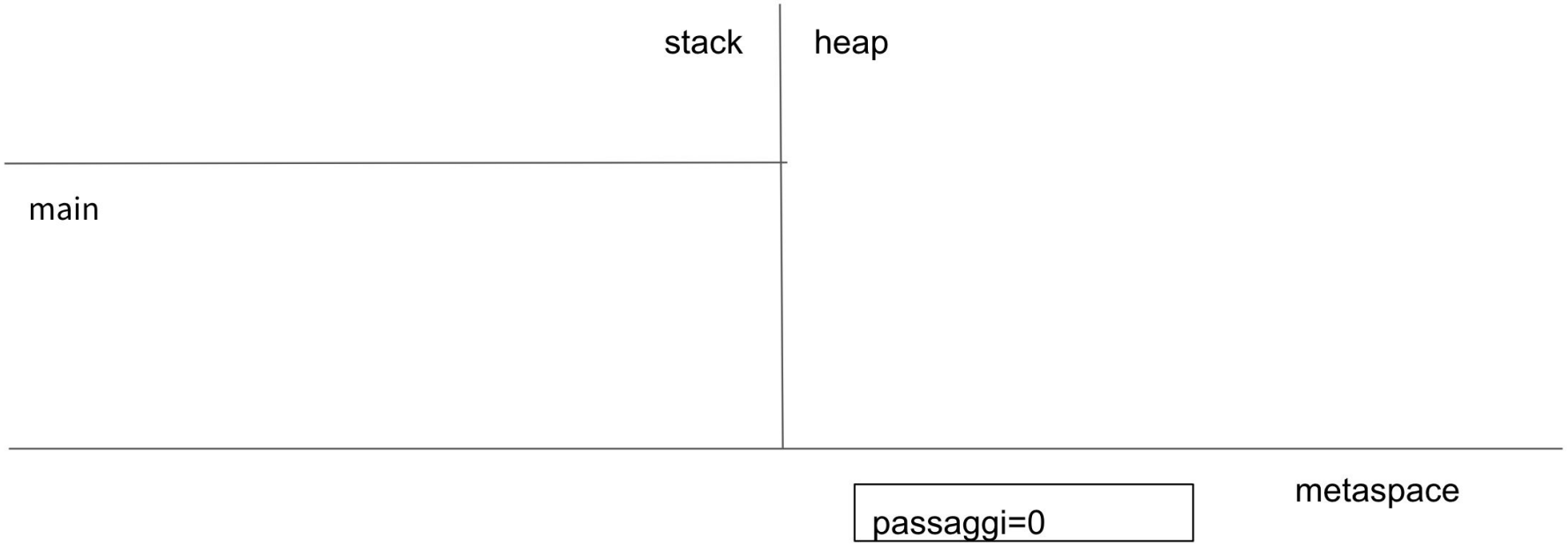


```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```

Analizziamo il main



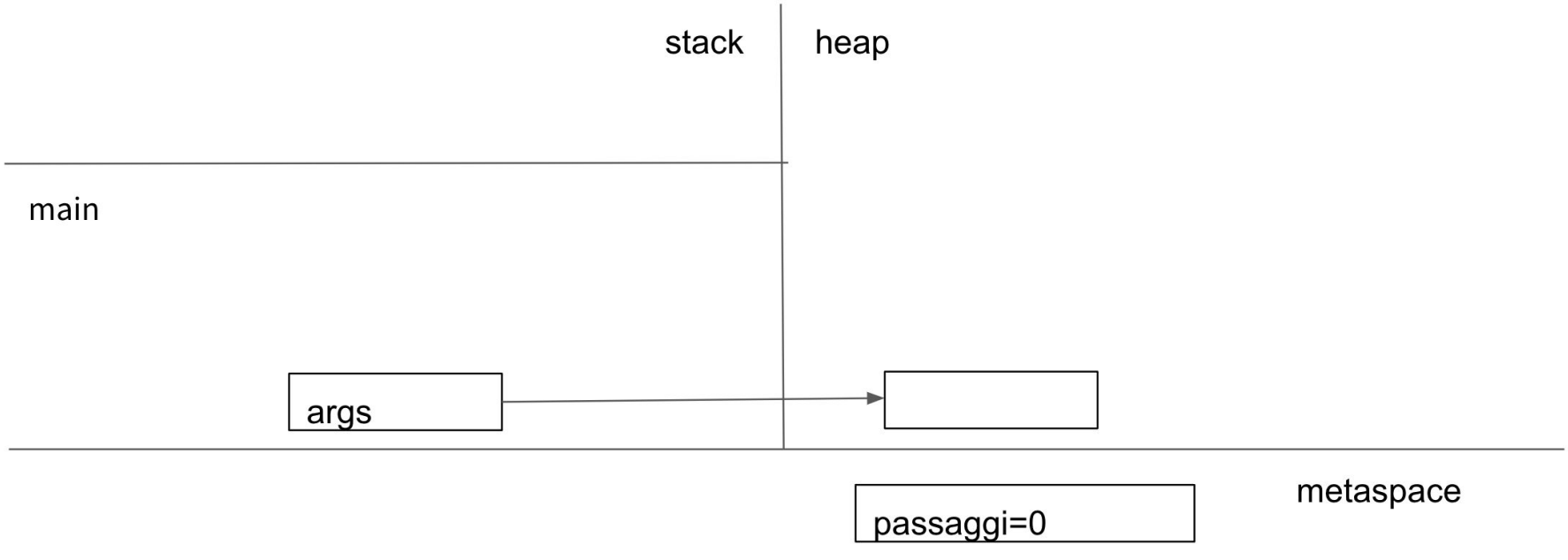
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



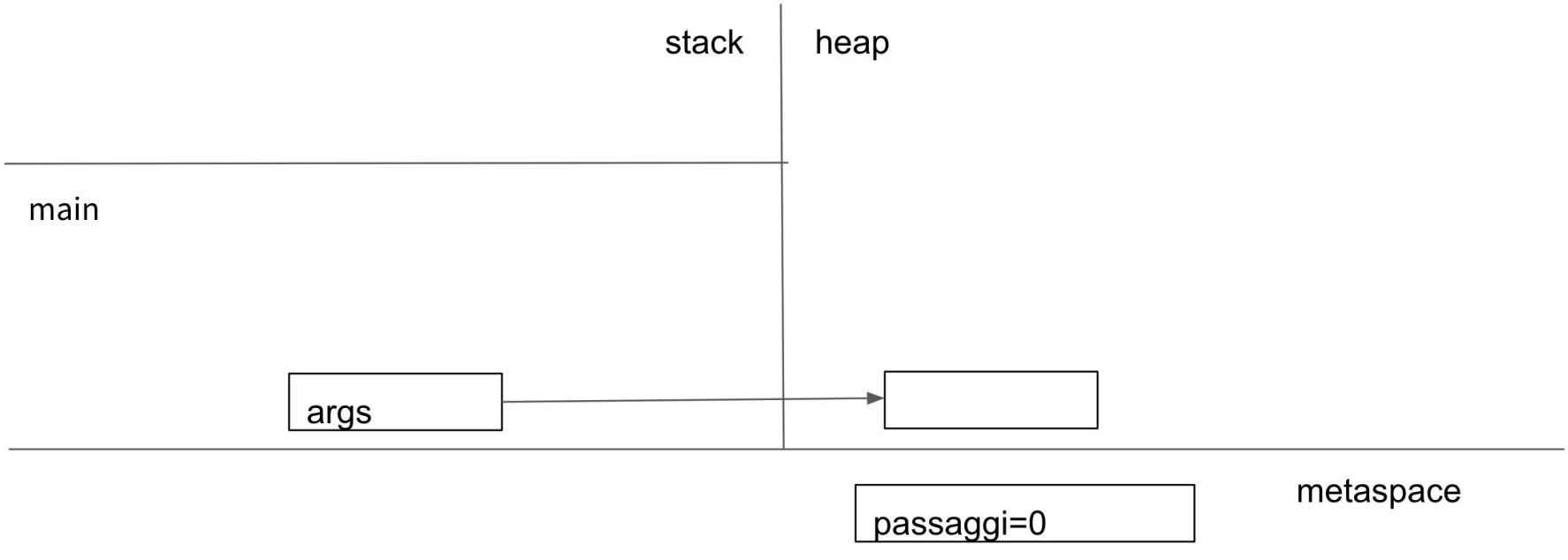
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



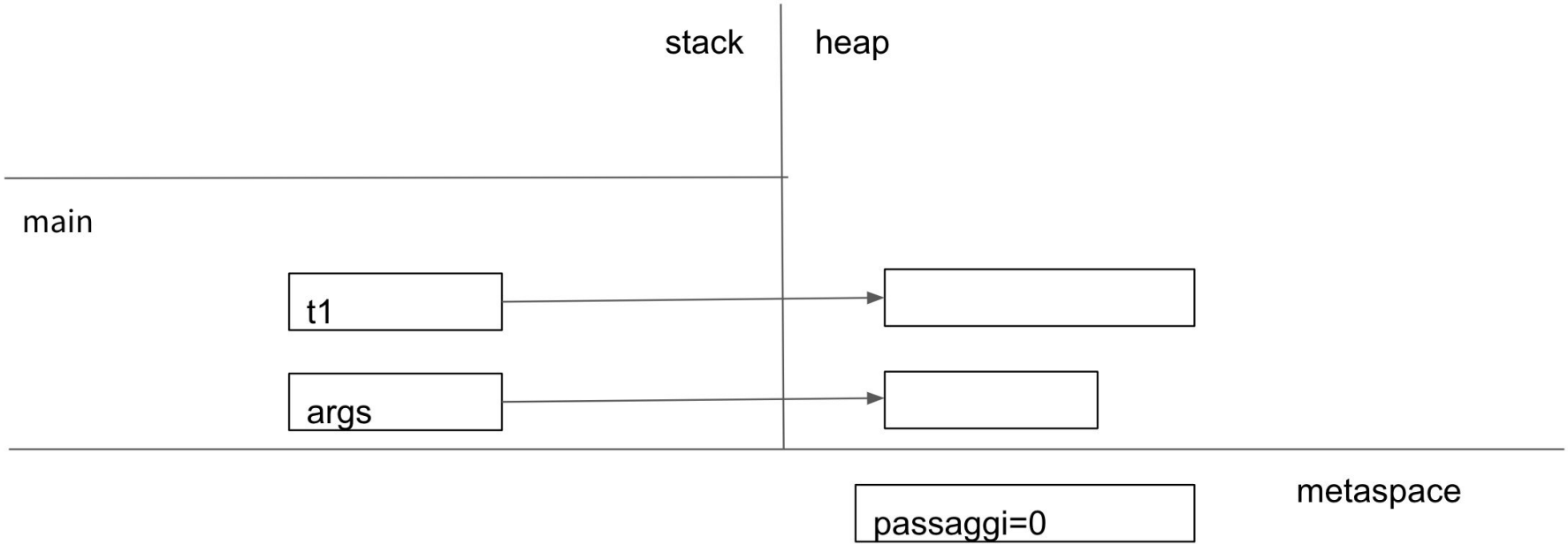
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



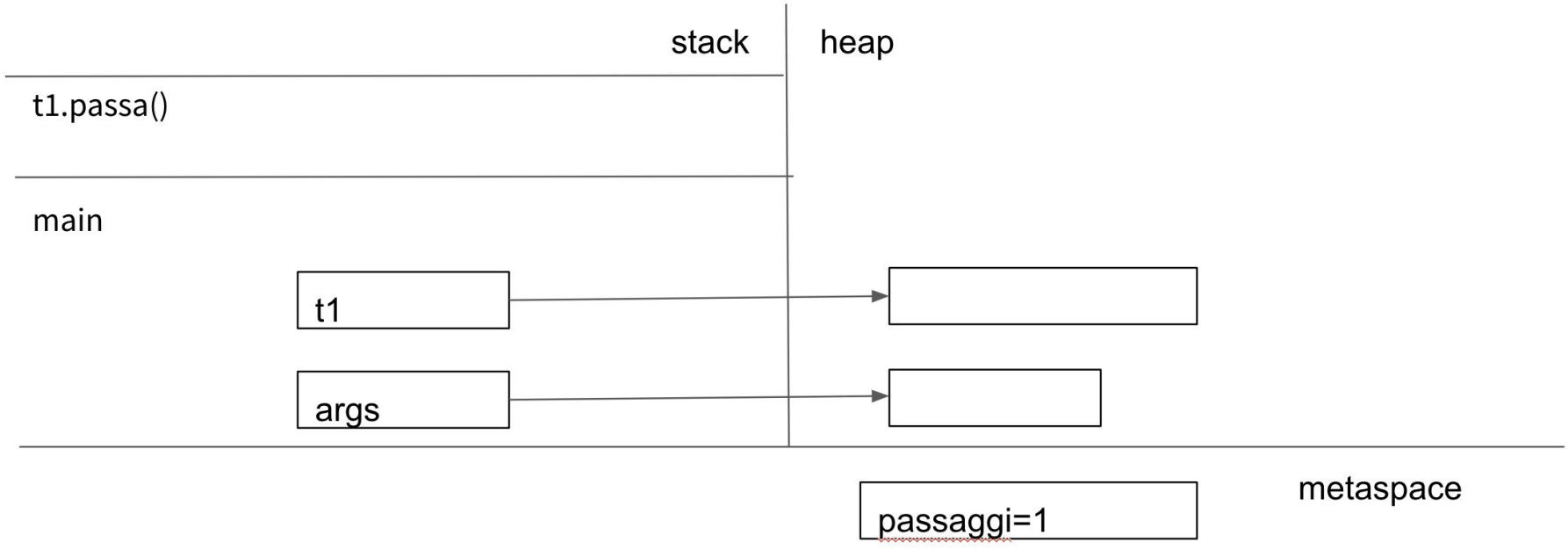
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



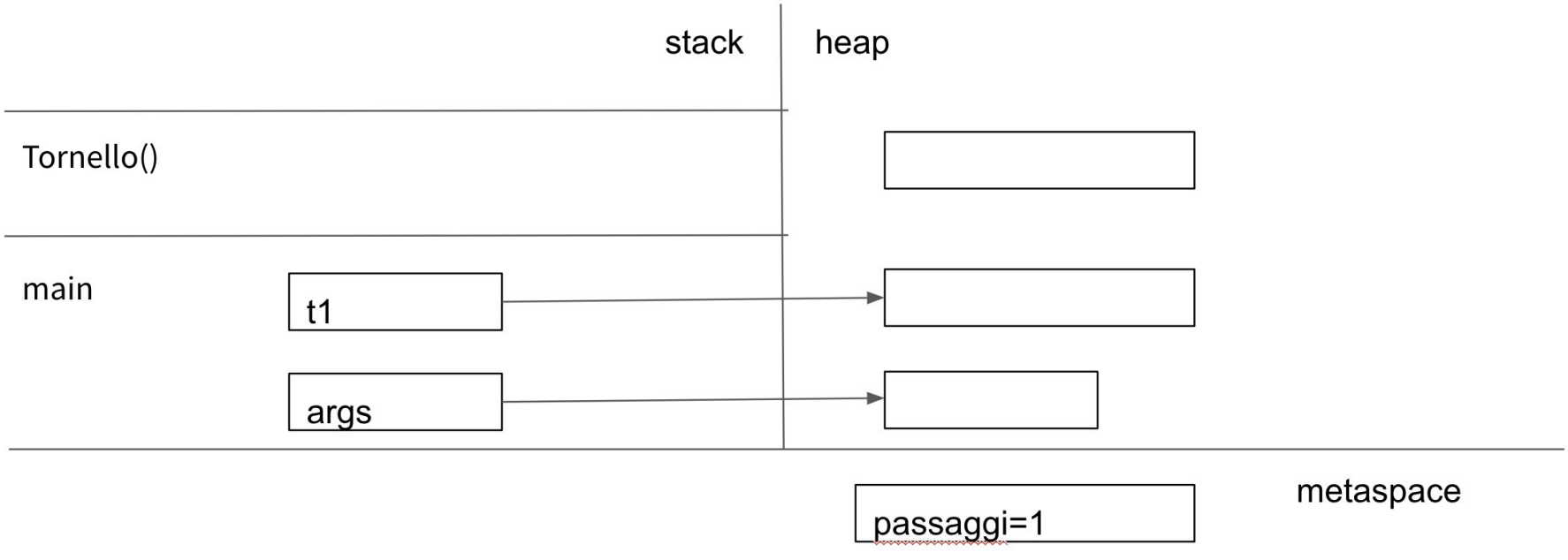
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



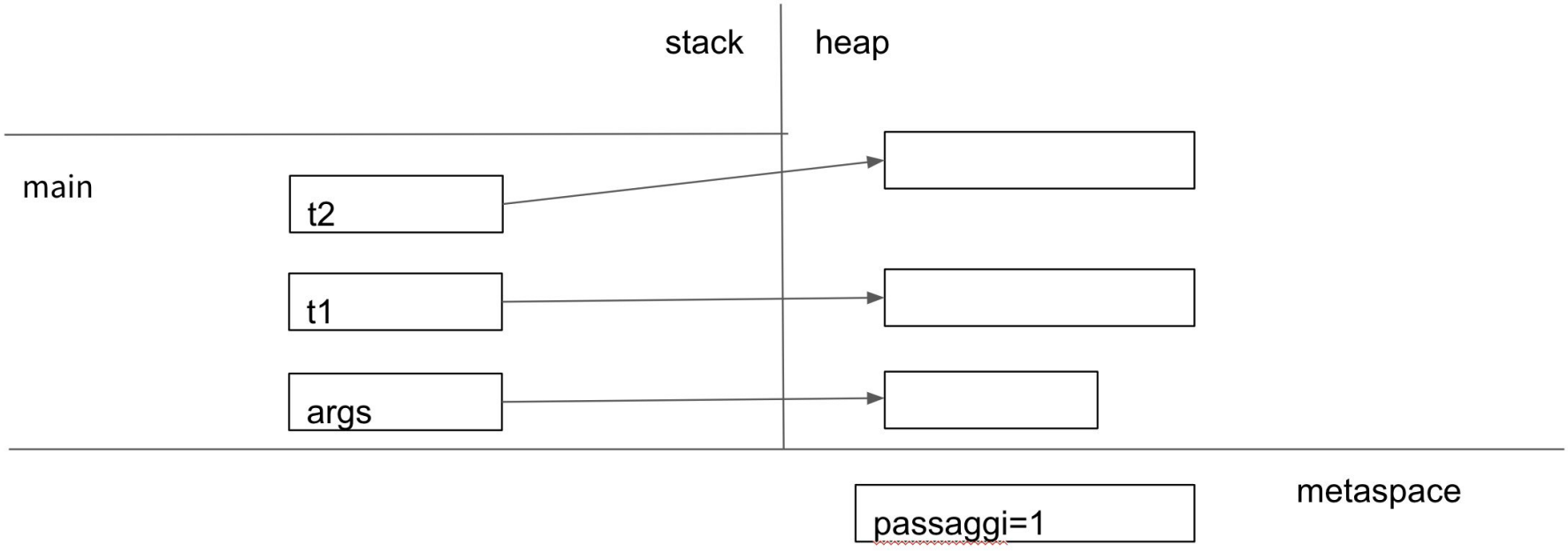
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



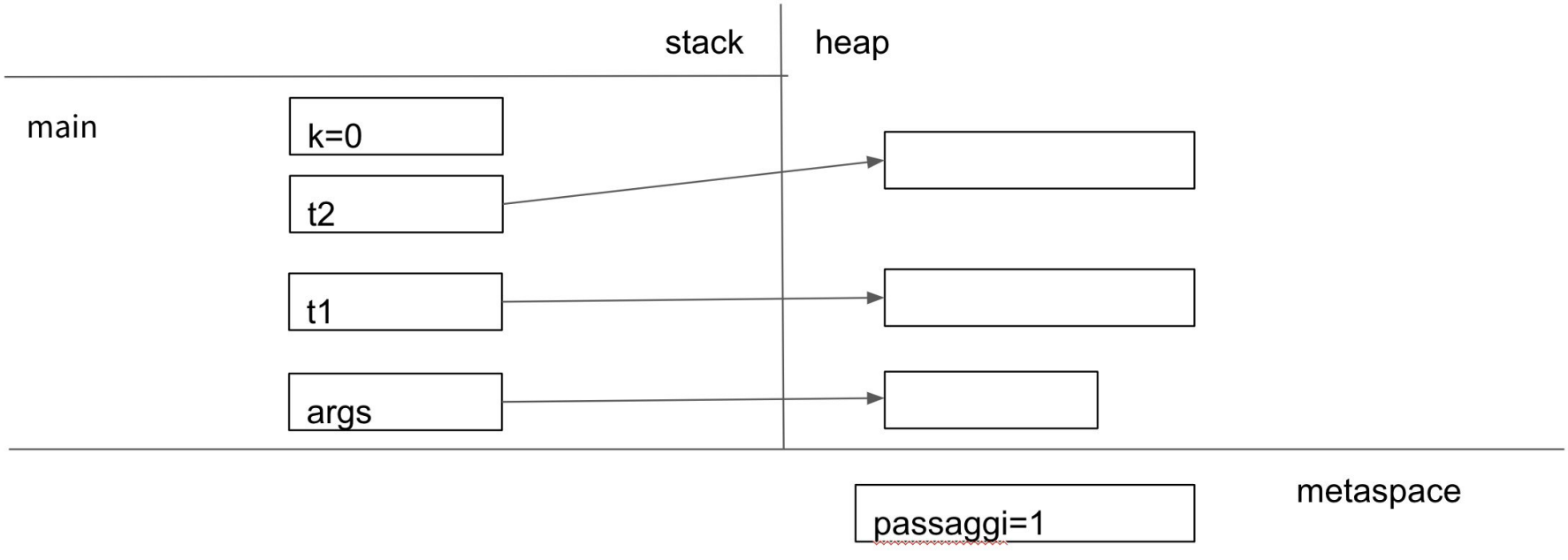
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



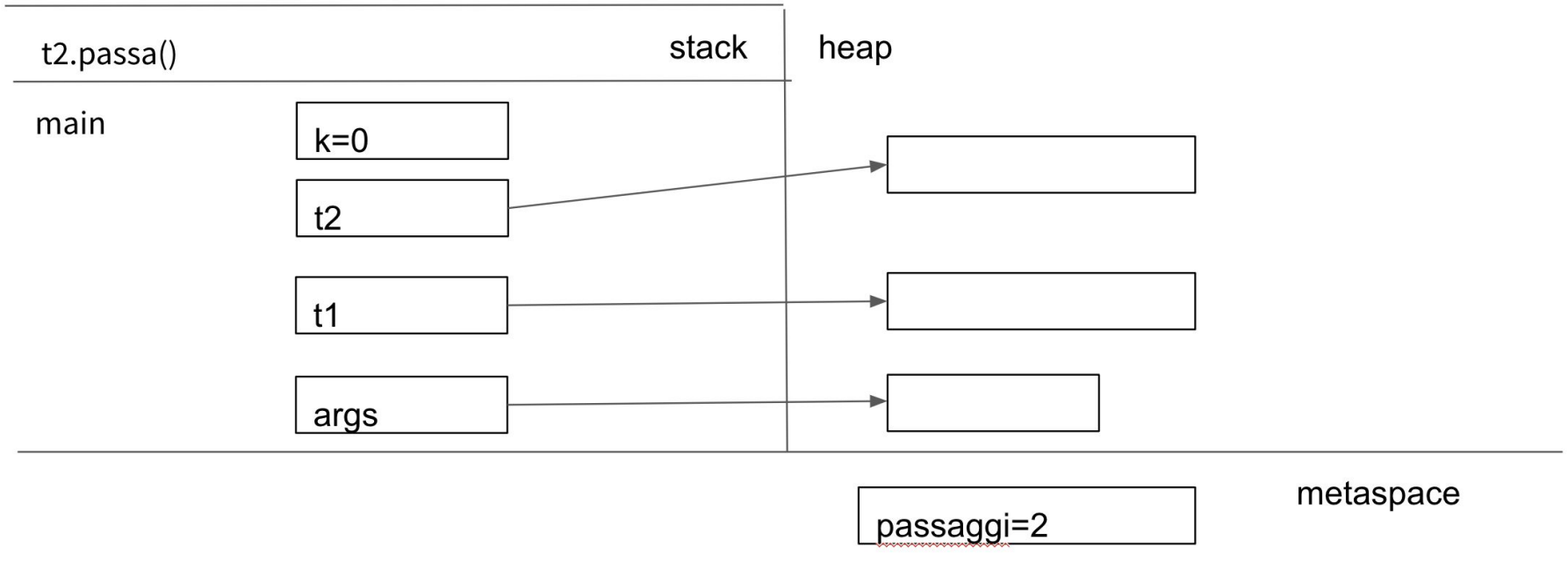
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



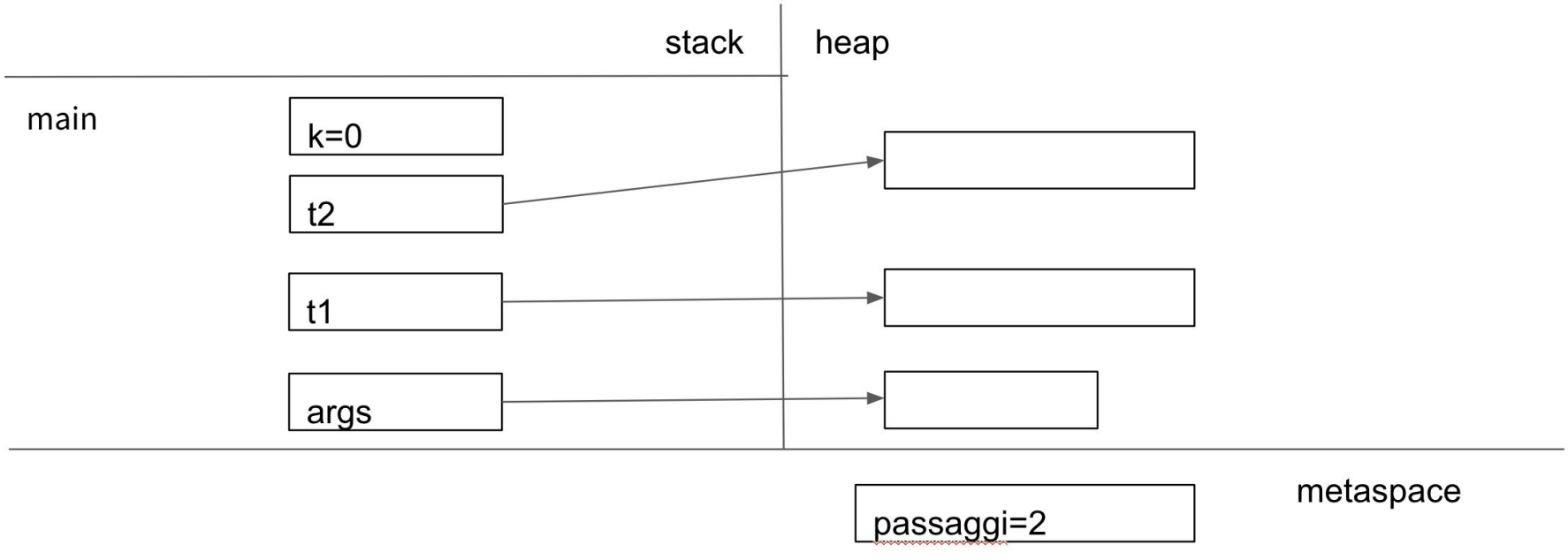
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



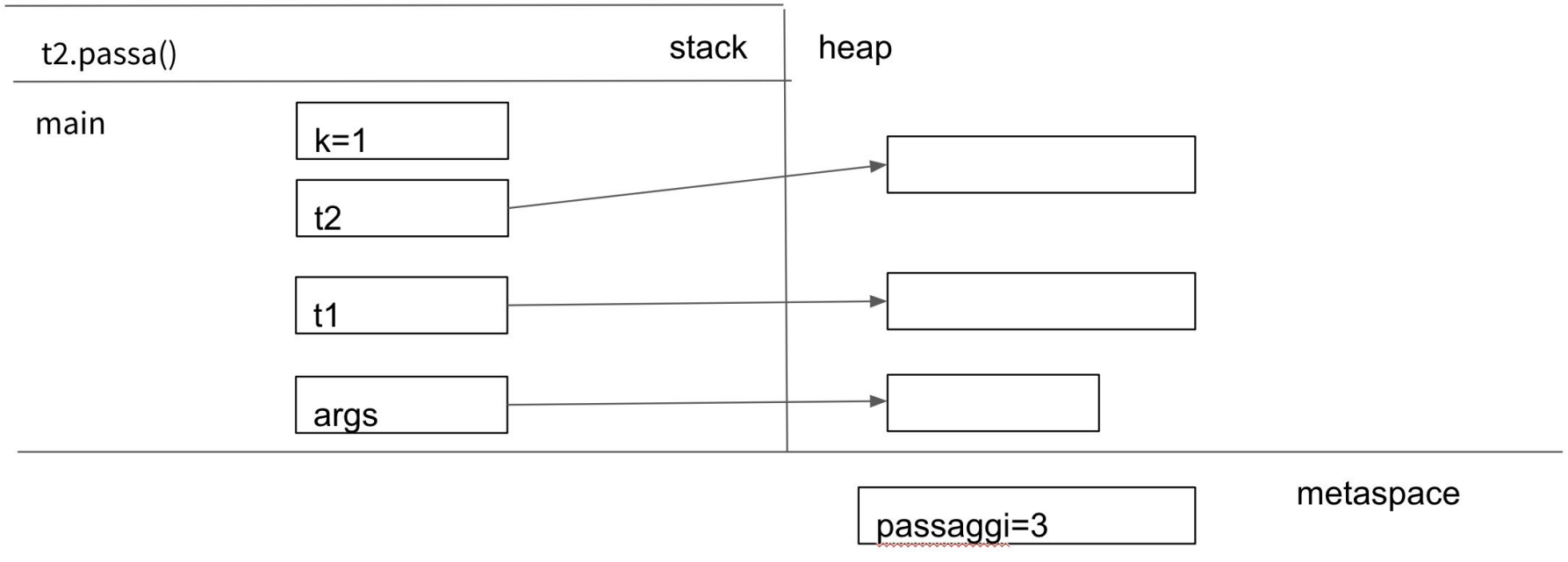
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```





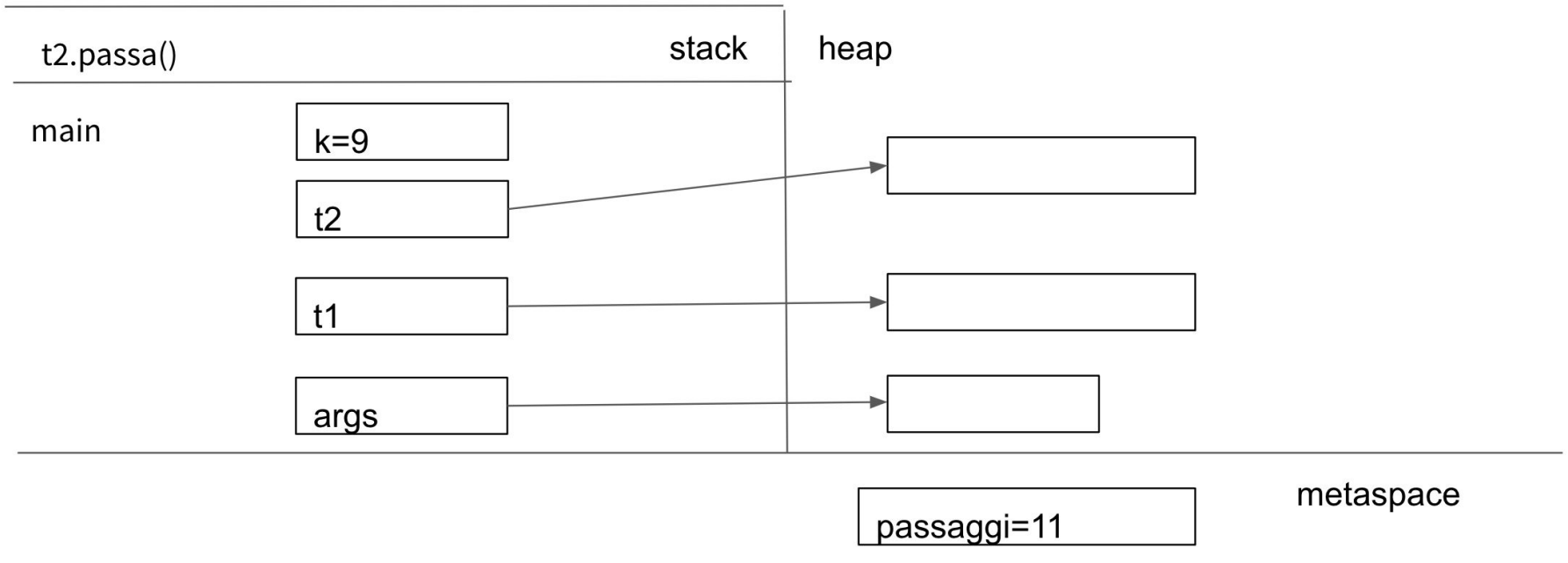
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



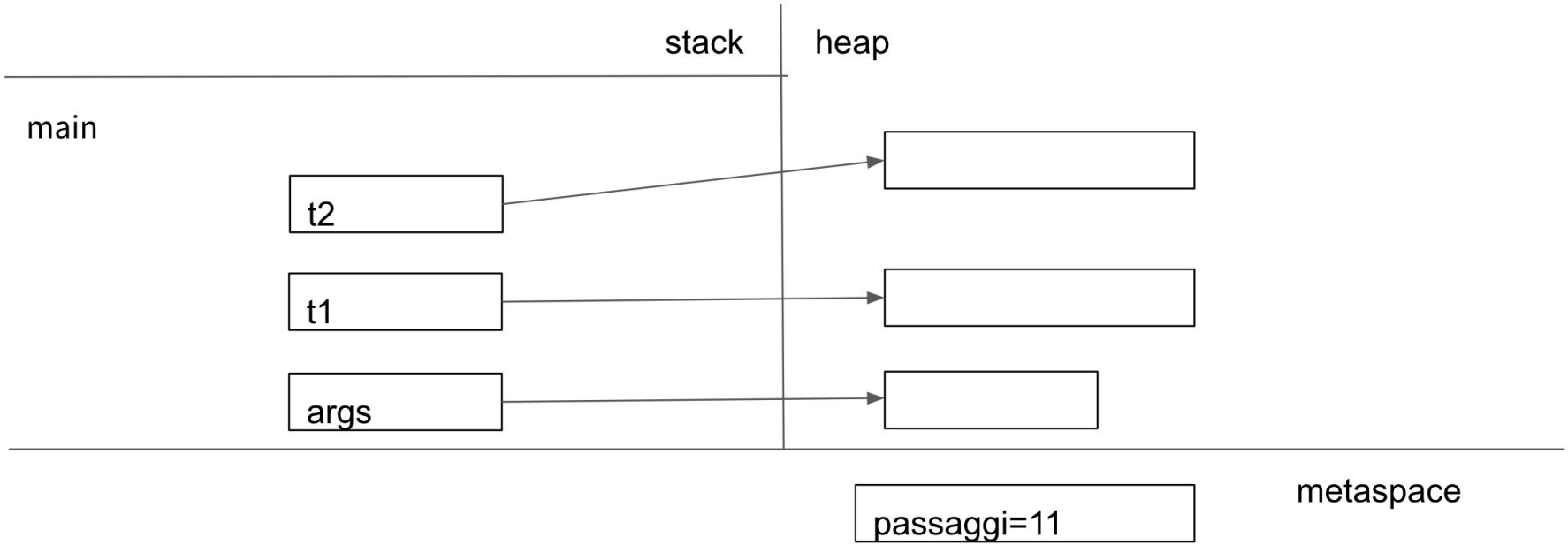
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



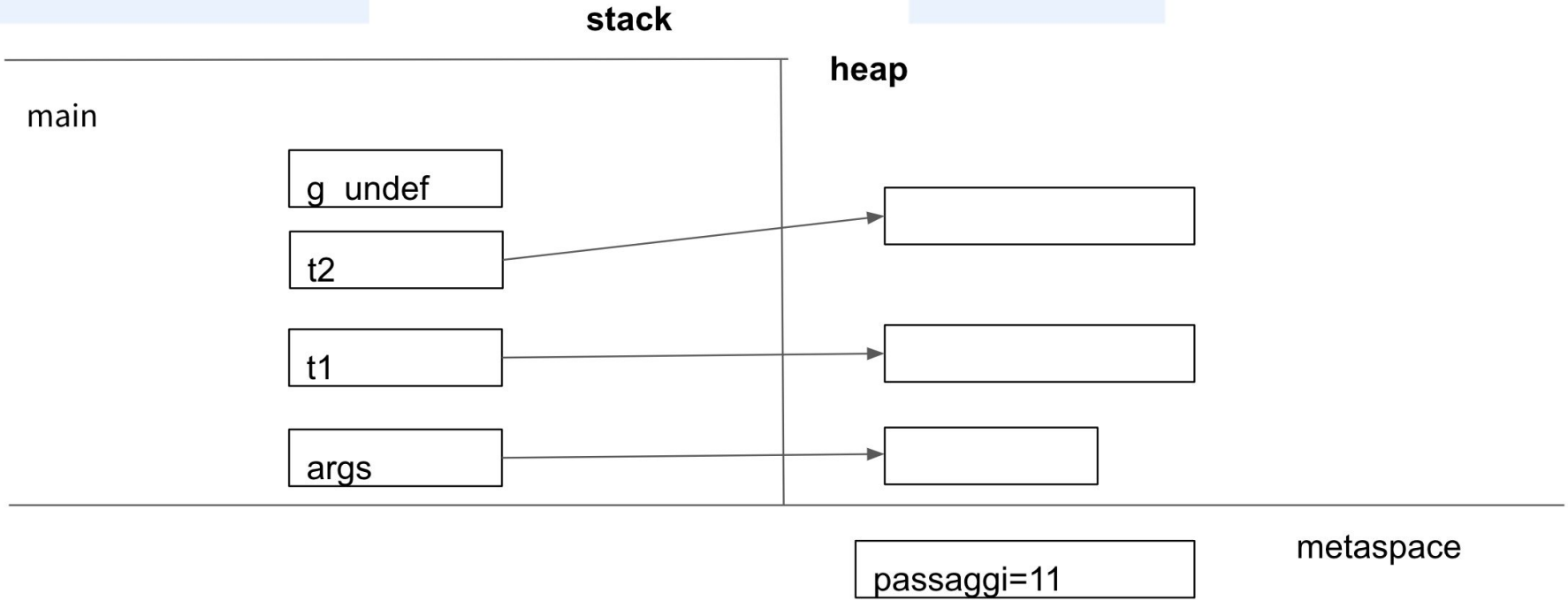
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```



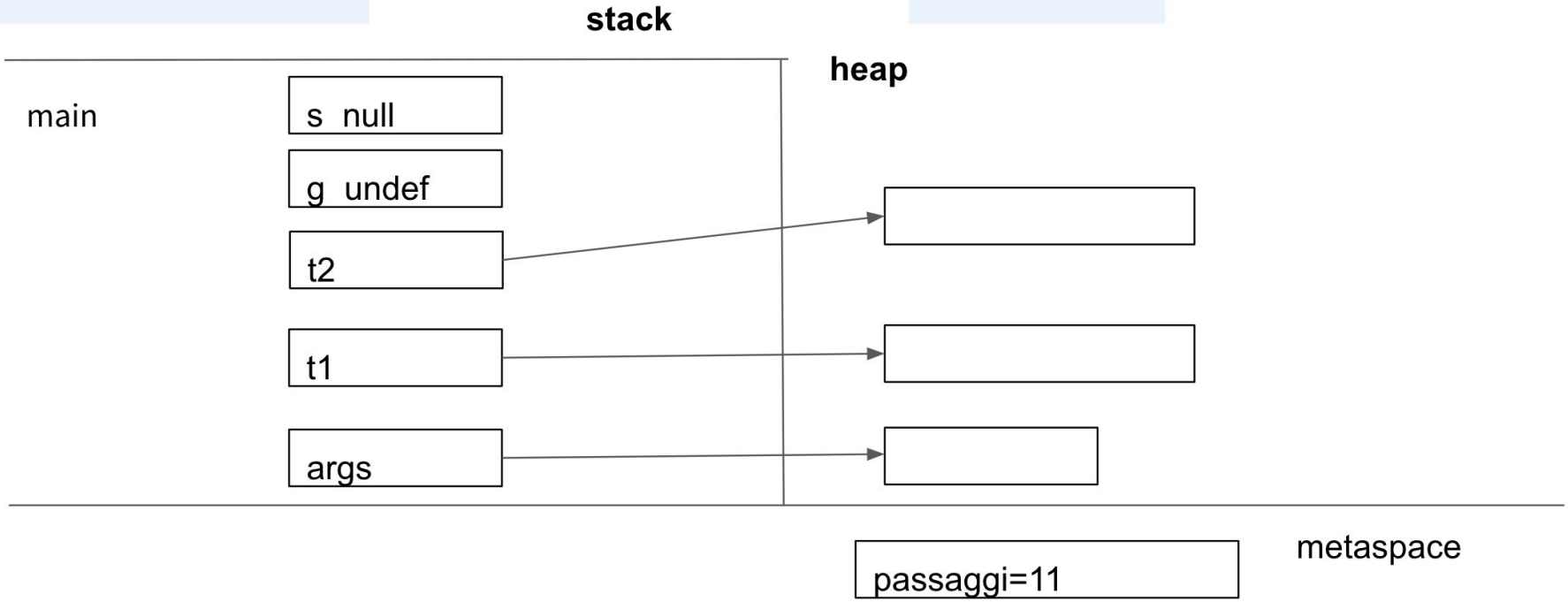
Tornello



```
public class Tornello {
    static private int passaggi;

    public void passa() {passaggi++;}
    public static int getPassaggi() {return passaggi;}

    public static void main(String[] args)
    {
        Tornello t1 = new Tornello();
        t1.passa();
        Tornello t2 = new Tornello();
        for (int k=0; k<10; k++) t2.passa();
        int g;
        String s=null;
        // fotografa lo stato della memoria
    }
}
```

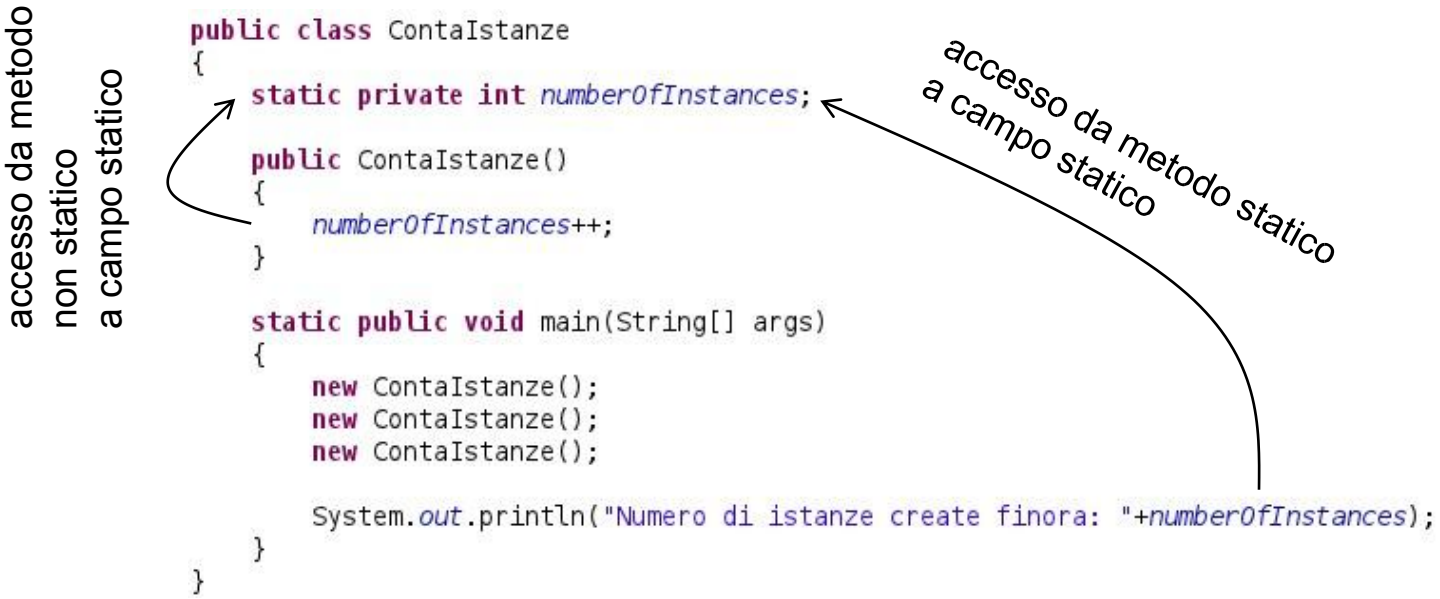


Esercizio: punti e segmenti

- Progettare una classe Punto per la rappresentazione di un punto nello spazio tridimensionale
- E una classe Segmento per rappresentare un segmento nello spazio tridimensionale
- Scrivere una classe di test che crei:
  - due oggetti della classe Punto con coordinate (1, 3, 8) e (4, 4, 7)
  - un oggetto della classe Segmento che rappresenti il segmento che unisce i due punti di cui sopra
- Raffigurare l'evoluzione dello stato della memoria, distinguendo tra stack , heap e metaspace

Metodi statici

- I metodi statici sono metodi di classe
- **NON** hanno accesso ai campi di istanza
- Ma hanno accesso ai campi di classe



Package

🎯Obiettivi

- ✅ incapsulamento
- ✅ uml
- ✅ metodi statici vs. d’istanza
- ✅ heap, stack, metaspace
- ☐ package

Package

- Le classi vengono inserite (categorizzate) in collezioni dette package
- Ogni package racchiude classi con funzionalità correlate
- Quando si utilizza una classe è necessario specificarne il package (come per Scanner, che appartiene al package java.util)
- Le classi che abbiamo usato finora (es. System, String) appartengono al package speciale java.lang
  - Questo package non deve essere specificato

- Le classi sono organizzate in package
- nomi dei package: sequenze di identificatori separati da punti

java.util

javax.swing

com.sun.misc
edu.sjsu.cs.cs151.alice

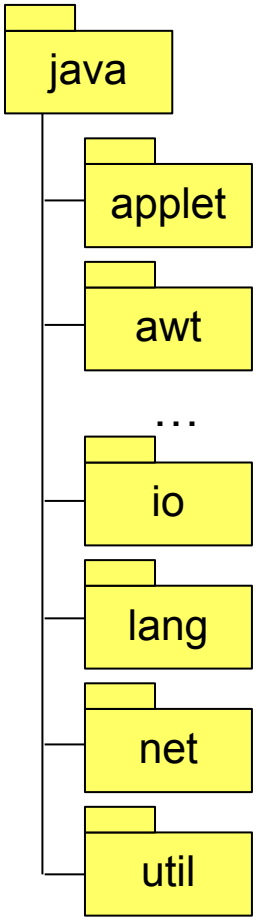
- Devono essere unici: convenzione reverse domain name
- Classi senza definizione di package sono nel "default package"
- Nome completo della classe = package + classe

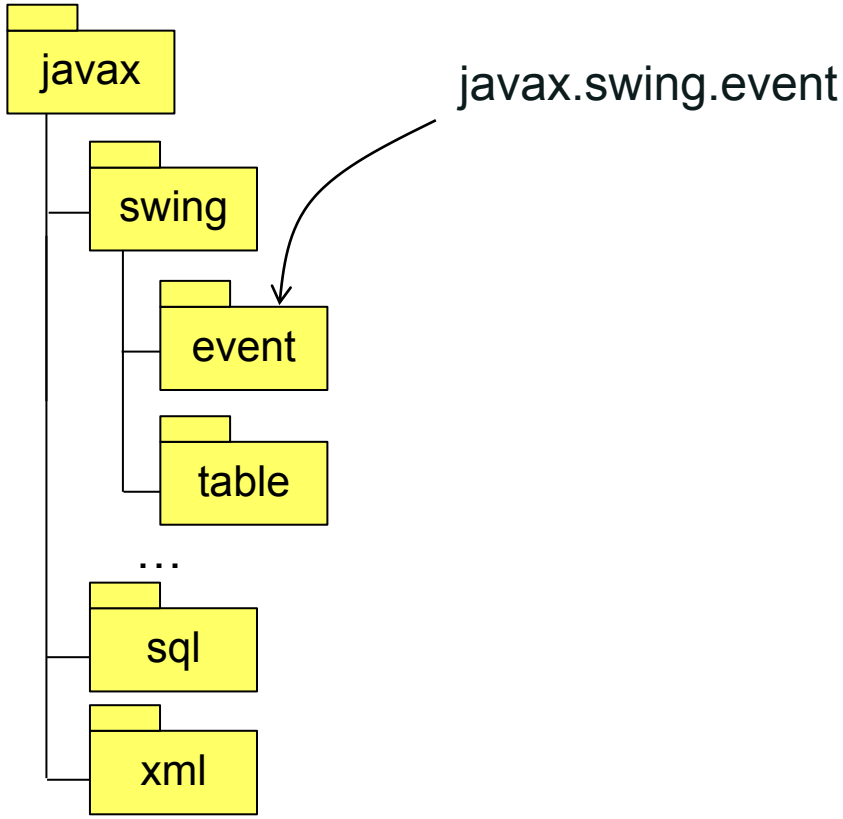
java.util.ArrayList

javax.swing.JOptionPane

Package standard

- Le API (Application Programming Interface) di Java sono organizzate in numerosi package





I nomi dei package devono coincidere con la struttura in cartelle

edu.sjsu.cs.sjsu.cs151.alice.Greeter

basedirectory/edu/sjsu/cs/sjsu/cs151/alice

Compilare sempre dalla cartella di base

javac edu/sjsu/cs/sjsu/cs151/alice/Greeter.java

Lo stesso per eseguire

java edu.sjsu.cs.cs151.alice.GreeterTest

La dichiarazione import

- Per evitare di specificare il package di una classe ogni volta che viene usata, è sufficiente importare la classe

```
import java.util.Scanner;

public class ChatBotNonCosiInterattivo
{
	public static void main(String[] args)
	{
		// crea uno Scanner per ottenere l'input da console
		Scanner input = new Scanner(System.in);

		System.out.println("Come ti chiami?");

		// leggere i caratteri digitati finche' non viene inserito
		// il carattere di nuova riga (l'utente preme invio)
		String nome = input.nextLine();

		System.out.println("Ciao "+nome+"!");
	}
}
```

La dichiarazione import

- Per evitare di specificare il package di una classe ogni volta che viene usata, è sufficiente importare la classe
- O l'intero package:

import java.util.*;

Attenzione: non è ricorsivo!

```
public class ChatBotNonCosiInterattivo
{
	public static void main(String[] args)
	{
		// crea uno Scanner per ottenere l'input da console
		Scanner input = new Scanner(System.in);

		System.out.println("Come ti chiami?");

		// leggere i caratteri digitati finche' non viene inserito
		// il carattere di nuova riga (l'utente preme invio)
		String nome = input.nextLine();

		System.out.println("Ciao "+nome+"!");
	}
}
```

Creazione di nuovi package

- I package sono rappresentati fisicamente da cartelle (String.class si trova sotto java/lang/)
- Una classe può essere inserita in un determinato package semplicemente
  - specificandolo all'inizio del file (parola chiave **package**)
  - posizionando il file nella corretta sottocartella
- Eclipse fa tutto questo per voi!

Esempio: la classe Triangolo

```
package it.navigli.geometry;

/**
 * La figura geometrica triangolo
 * @author navigli
 *
 */
public class Triangolo
{
	/**
	 * Base del triangolo
	 */
	private double base;

	/**
	 * Altezza del triangolo
	 */
	private double altezza;

	/**
	 * Costruttore del triangolo
	 * @param base base del triangolo
	 * @param altezza altezza del triangolo
	 */
	public Triangolo(double base, double altezza)
	{
		this.base = base;
		this.altezza = altezza;
	}

	/**
	 * Restituisce l'area della figura
	 * @return l'area
	 */
	public double getArea()
	{
		return base*altezza/2.0;
	}
}
```

Triangolo.java si deve trovare nella cartella it/navigli/geometry

Commento Javadoc per la classe

Javadoc per un campo

Javadoc per il costruttore

Parametri Javadoc per il costruttore

Javadoc per un metodo

Riferimento all' oggetto costruito

Javadoc per il valore restituito

🎯Obiettivi

- ✅ incapsulamento
- ✅ uml
- ✅ metodi statici vs. d’istanza
- ✅ heap, stack, metaspace
- ✅ package

javadoc

Commenti come documentazione

Delimitati da /** ... */

Prima frase = riassunto

@param parametri

@return valore di ritorno

Javadoc utility estrae HTML

il risultato

https://docs.oracle.com/en/java/javase/21/docs/api/index.html

Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.